

1. Introduction

A comprehensive understanding of vehicle dynamics is essential for improving performance, safety and driving quality in both road cars and high-performance vehicles. Lateral and vertical dynamics represent two complementary domains whose combined analysis enables a more complete optimisation of vehicle behaviour. On one hand, lateral dynamics analysis is essential when evaluating vehicle's performance during cornering. To this end, simplified models can be exploited to assess the key aspects of lateral response, such as sideslip angles and lateral forces developed by tyres. In parallel, the analysis of vertical suspension dynamics plays a crucial role in determining both comfort and handling performance. Road excitation reduces tyres' ability to generate lateral forces because it induces variations in the normal load, thereby worsening handling behaviour. At the same time, it causes vibrations transmitted to the chassis, which negatively impact comfort (Genta and Morello, 2020). For this reason, suspension models are commonly employed to optimise parameters such as damping coefficient, eventually providing an effective balance between comfort and handling.

This study provides an analysis of lateral and vertical vehicle dynamics, with the aim of addressing reliability and fidelity of several models, while also proposing optimisation strategies to enhance vehicle performance. Lateral dynamics is analysed on a Nissan GTR-R35, while vertical dynamics on a Ford Granada. Refer to Appendix A and B for details.

2. Bicycle model and handling analysis

The bicycle model offers an effective means of describing vehicle's handling characteristics. By reducing a four-wheel, two-axle vehicle as an equivalent system with two-wheels connected by a rigid link (Talukdar and Kulkarni, 2011), the model captures the essential features of the lateral response while remaining computationally efficient. This simplicity makes it highly suitable for early-stage vehicle development (Sondkar *et al.*, 2025) as well as for ADAS and autonomous driving applications for trajectory planning, as it captures enough information about vehicle's lateral behaviour to enable effective control (Polack *et al.*, 2017). Nevertheless, the assumptions underlying the bicycle model, such as constant longitudinal speed, absence of load transfer and rigid suspensions (Milliken and Milliken, 1995), restrict its reliability to specific conditions. As a result, the bicycle model cannot reliably assess vehicle's lateral behaviour in sport driving conditions, where large accelerations occur (Genta and Morello, 2020).

2.1. Description and Discussion of Models

The underlying equations of motion describing the bicycle model are reported in Equation 1 (Novara, 2025), while a schematic representation is provided in Figure 1. Details about models' implementation are available in Appendix D.

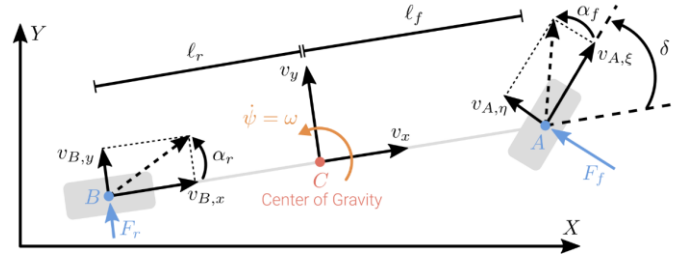


Figure 1: Bicycle model (Kim, 2023).

$$\begin{cases} \dot{X} = v_x \cos \psi - v_y \sin \psi \\ \dot{Y} = v_x \sin \psi + v_y \cos \psi \\ \dot{\psi} = \omega_\psi \\ \dot{v}_x = v_y \omega_\psi \\ \dot{v}_y = -v_x \omega_\psi + \frac{F_{yf} + F_{yr}}{m} \\ \dot{\omega}_\psi = \frac{a F_{yf} - b F_{yr}}{J} \end{cases} \quad (1)$$

2.1.1. Linear

The simplified linear bicycle model features a linear steady-state relationship between lateral-force and side-slip angle, as reported in Equation 2. Different configurations are simulated to analyse vehicle's behaviour in understeer, oversteer and neutral-steer conditions. To accomplish this, three vehicle models were designed, each differing in the position of vehicle's centre of gravity, which determines the load distribution between front and rear axles, and thus the steering characteristic. For each configuration, the model was firstly implemented in ADAMS and consequently in MATLAB/Simulink, to validate the obtained results using analytical formulations.

$$\begin{cases} F_{yf} = -C_f \cdot \alpha_f \\ F_{yr} = -C_r \cdot \alpha_r \end{cases} \quad (2)$$

Side-slip angles are computed using linearized formulations depending on the state variables, as reported in Equation 3 (Amati, 2025). No dependence on lateral dimensions is present, as the bicycle model is developed exclusively in the longitudinal direction.

$$\begin{cases} \alpha_f = \frac{v_y + a\dot{\psi}}{v_x} - \delta \\ \alpha_r = \frac{v_y - b\dot{\psi}}{v_x} \end{cases} \quad (3)$$

2.1.1.1 Understeer

Since understeer represents the safest vehicle behaviour, passenger cars are typically designed to feature this characteristic (Tonoli, 2023). Therefore, understeer model is obtained with centre of gravity position of the real vehicle, corresponding to a 54% front and 46% rear load-distribution. Figure 2 shows the understeering bicycle model implemented in ADAMS.

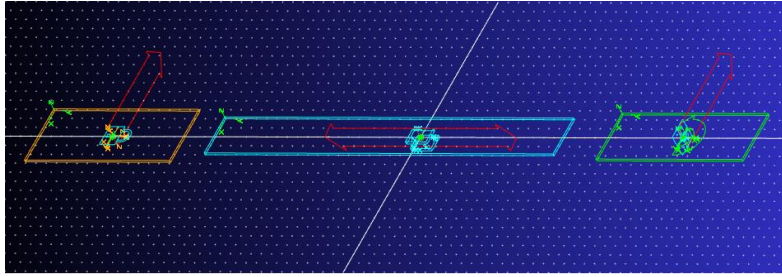


Figure 2: Understeering ADAMS bicycle model.

2.1.1.2 Oversteer

To analyse oversteer, the model was modified by shifting the centre of gravity rearward, assuming a 70% rear and 30% front load-distribution. This change alters the cornering stiffness, which was therefore updated using the procedure explained in Appendix B. Figure 3 depicts the oversteering bicycle model, highlighting the rearward-shifted CG.

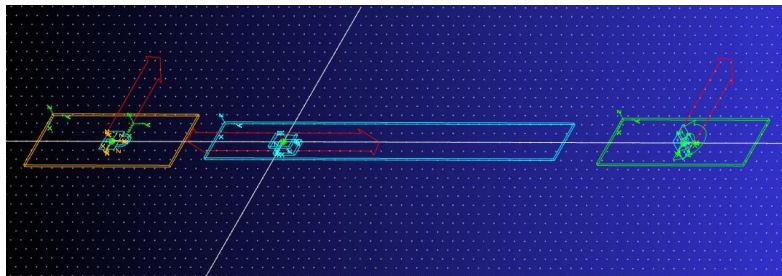


Figure 3: Oversteering ADAMS bicycle model.

2.1.1.3 Neutral Steer

The neutral steer condition was achieved with a 61.5% rear and 38.5% front load distribution. These values were determined through a trial-and-error procedure, where curvature gain was evaluated for different load distributions. The final configuration was selected to match the curvature gain typical of neutral steer conditions, as discussed later.

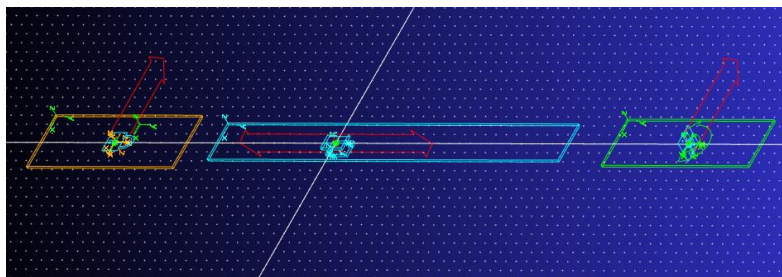


Figure 4: Neutral steering ADAMS bicycle model.

2.1.2. Nonlinear

To improve model fidelity, the bicycle model was extended to implement tyre nonlinear characteristics. The nonlinear lateral force-slip angle relationship was represented using a normalized spline featuring the linear portion in the 0÷1 range on both axes, then rescaled according to cornering stiffness values.

Spline results are shown in Figure 5. MATLAB code used to generate these splines is provided in Appendix H.

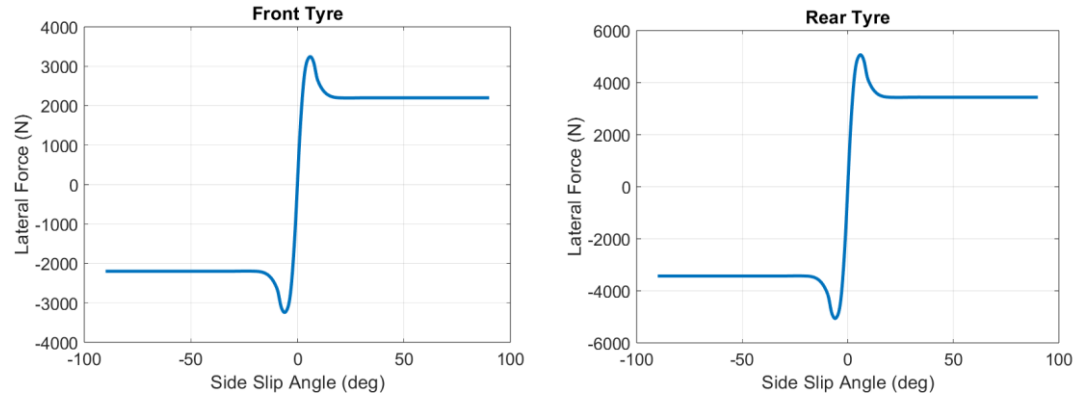


Figure 5: Lateral force-side slip angle LUTs spline.

2.1.3 Transient

The model accounting for tyre's transient behaviour is developed in MATLAB/Simulink, incorporating Equation 4 to account for the dynamic buildup of lateral forces (Tonoli, 2023).

$$\dot{F}_{y,t} = \frac{V_x}{L_y} (F_{y,ss} - F_{y,t}) \quad (4)$$

L_y represents the relaxation length, which is used to characterize the transient tyre property (Loeb *et al.*, 1990). A reasonable value is $L_y = 0.045 \text{ m}$ (Tonoli, 2023). $F_{y,ss}$ represents the steady-state lateral force, evaluated through the LUT described in Section 2.1.2, while $F_{y,t}$ represents the lateral force generated when the transient response of the tyre is considered.

2.1.4 Double-Track

The double-track model accounts for axle load transfer during cornering and its effect on cornering stiffness, thereby improving model fidelity, as body roll cannot be neglected in real-world scenarios (Milliken and Milliken, 1995). The vertical load is evaluated as the algebraic sum of static load and load transfer contribution. The static load corresponds to half of axle load, while the total load transfer contribution, evaluated using Equation 5, is distributed 60% to the front and 40% to the rear. This assumption reflects typical road vehicles design, where a greater front roll stiffness is adopted to increase understeer, thereby improving vehicle safety (Tonoli, 2023).

$$\Delta F_z = F_y \frac{h_G}{t} \quad (5)$$

F_y is the total lateral force generated by the four tyres to keep the vehicle on its intended trajectory, while h_G denotes the centre of gravity height.

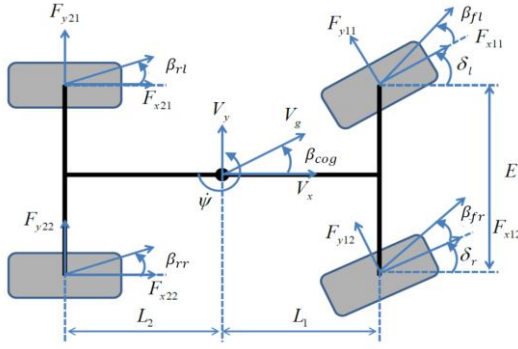


Figure 6: Double track model (Jiang, Victorino and Charara, 2016).

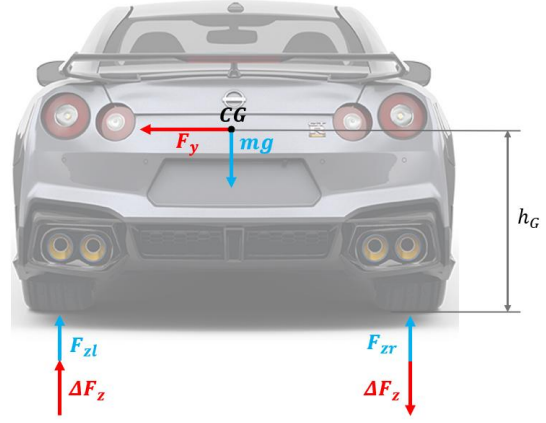


Figure 7: Simplified representation of forces during cornering.

The side-slip angles evaluation includes the effect of track width and different steering angles of front tyres resulting from positive Ackerman steering geometry, which makes the inner tyre steer at a larger angle than the outer one. Consequently, four separate equations, one for each tyre, are required to compute the side-slip angles, as reported in Equation 6 (Amati, 2025). The positive Ackerman steering condition is accounted for through Equation 7, which evaluates the right and left tyre steering angle starting from δ . The latter maintains the same time evolution used in the previous simulations, however in this case it represents the average steering angle between right and left wheels (Tonoli, 2023).

$$\left\{ \begin{array}{l} \alpha_{f,R} = \frac{v_y + a\dot{\psi}}{v_x + r \cdot \frac{t_f}{2}} - \delta_R \\ \alpha_{f,L} = \frac{v_y + a\dot{\psi}}{v_x - r \cdot \frac{t_f}{2}} - \delta_L \\ \alpha_{r,R} = \frac{v_y - b\dot{\psi}}{v_x + r \cdot \frac{t_f}{2}} \\ \alpha_{r,L} = \frac{v_y - b\dot{\psi}}{v_x - r \cdot \frac{t_f}{2}} \end{array} \right. \quad (6)$$

$$\left\{ \begin{array}{l} \delta_R = \frac{L}{R + \frac{t}{2}} \\ \delta_L = \frac{L}{R - \frac{t}{2}} \\ R = \frac{L}{\delta} \end{array} \right. \quad (7)$$

2.2. Results and Discussion

This section compares the results obtained from ADAMS and/or MATLAB/Simulink for linear, nonlinear, transient and double-track simulations. ADAMS results appear in the left column, while MATLAB/Simulink results in the right one. Both software produce nearly identical trends and steady-state values for the considered variables, cross validating the simulations.

2.2.1. Linear

The reported results for understeer, oversteer and neutral-steer configurations are obtained from simulations performed at $V_x = 10 \text{ m/s}$.

2.2.1.1 Understeer

The vehicle's understeering behaviour is assessed from side-slip angles. As shown by Figure 8, the front side-slip angle is always greater than the rear one, meaning that front wheel slides more, pushing the vehicle out of its trajectory. To maintain the same trajectory and curvature radius, the steering angle should increase, characteristic of understeering vehicles (Gillespie, 2021). The larger front slip angle also produces higher front lateral forces, although the difference is moderated by the rear's higher cornering stiffness.

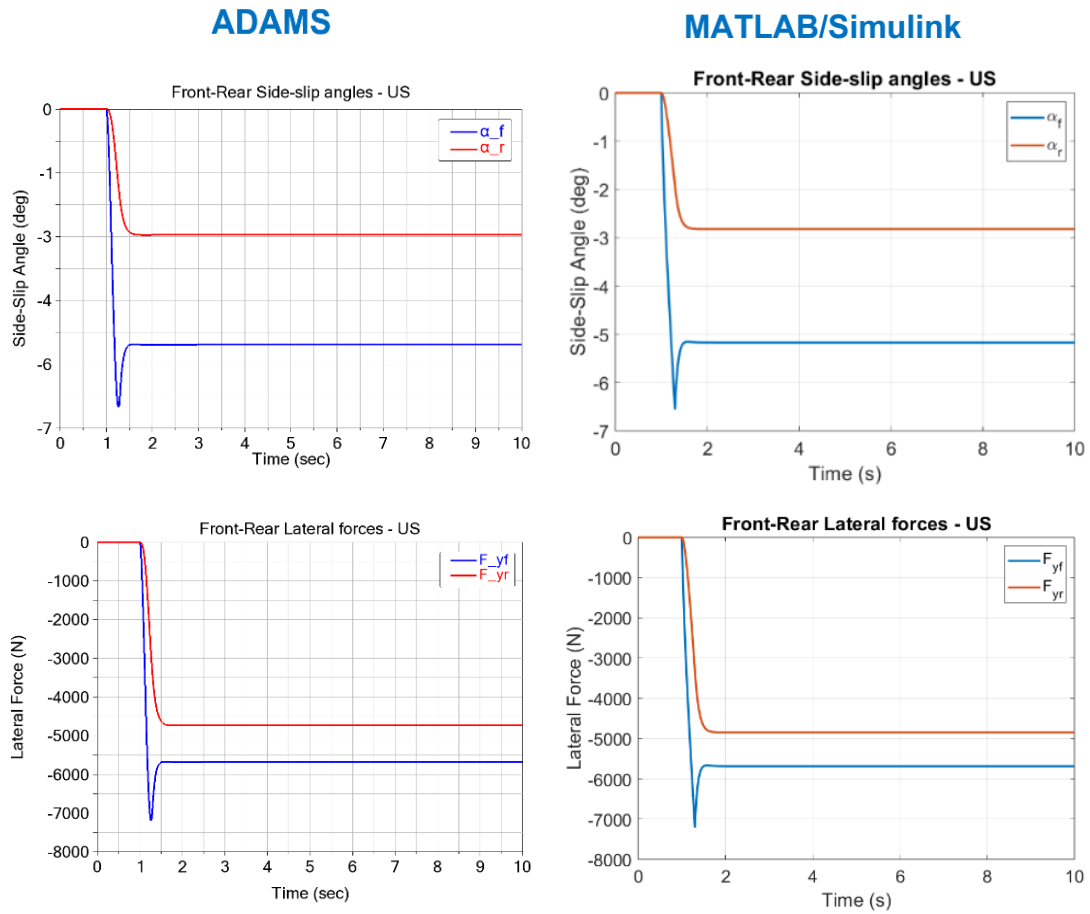


Figure 8: Understeer vehicle results.

2.2.1.2 Neutral Steer

A neutral-steer vehicle has identical slip angles at front and rear wheels, as shown in Figure 9. Under this condition, the vehicle does not deviate from its intended path, since any lateral force at its CG produces the same change in slip angle at both axles (Gillespie, 2021).

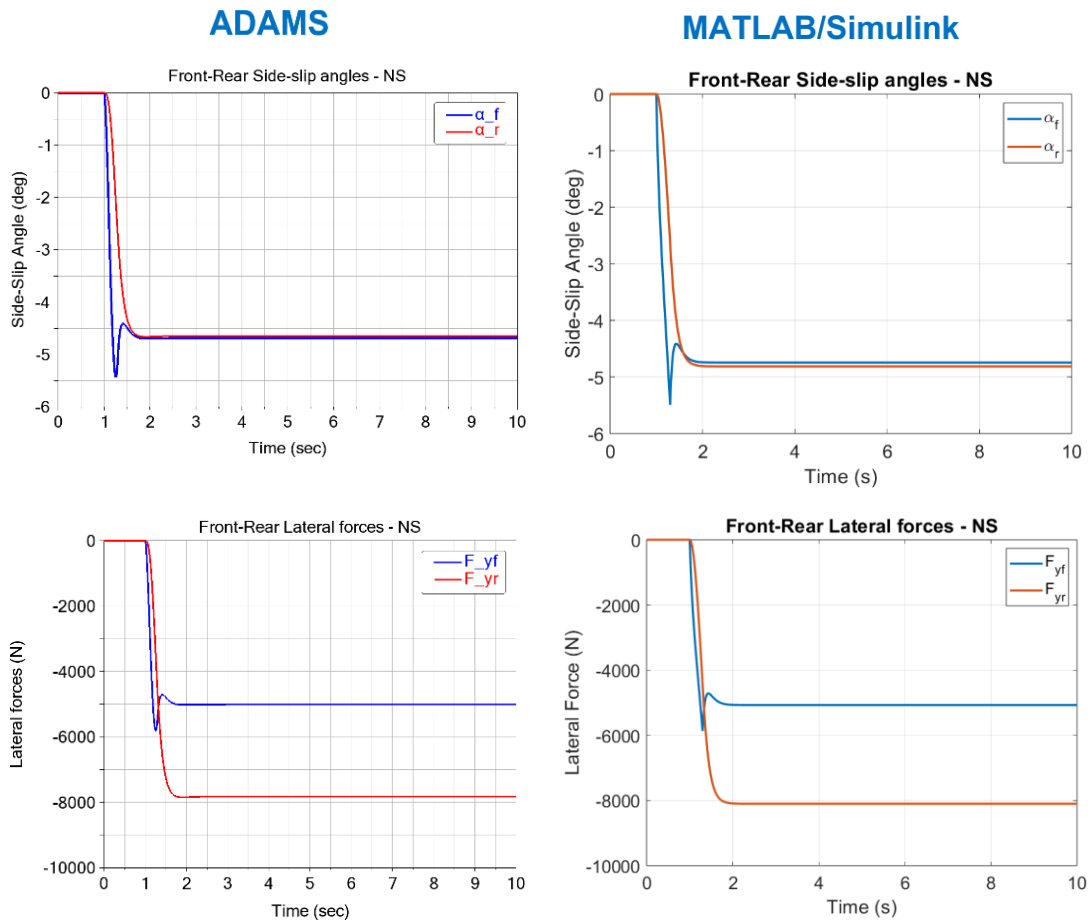
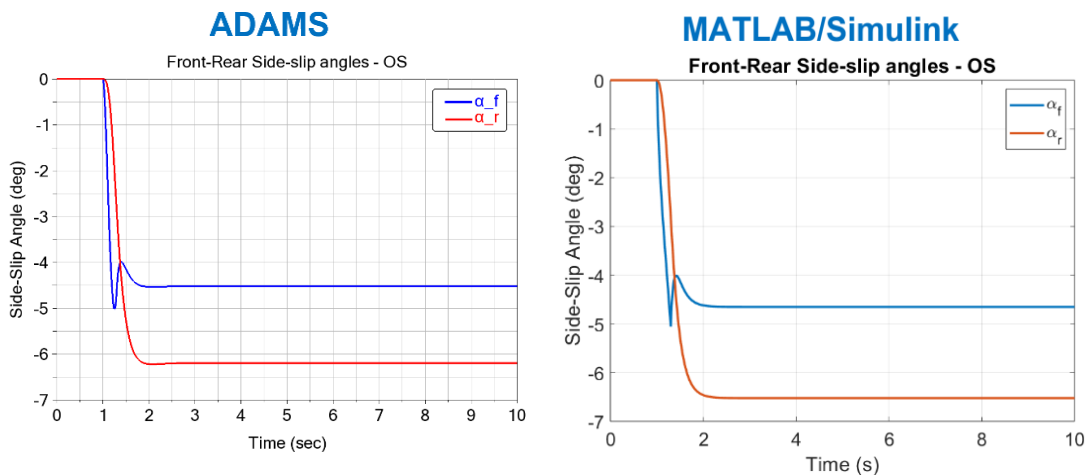


Figure 9: Neutral-steer vehicle results.

2.2.1.3 Oversteer

Oversteer vehicles are characterized by larger front slip angles, as shown in Figure 10, causing the rear to drift outward and forcing the front wheel to turn inward, eventually reducing the turning radius (Gillespie, 2021). Lateral forces show a larger difference between rear and front wheels compared with the understeering case because the higher rear cornering stiffness, which in understeer helps counteract large front slip angles, in oversteer amplifies the disparity between front and rear forces, further increasing the gap.



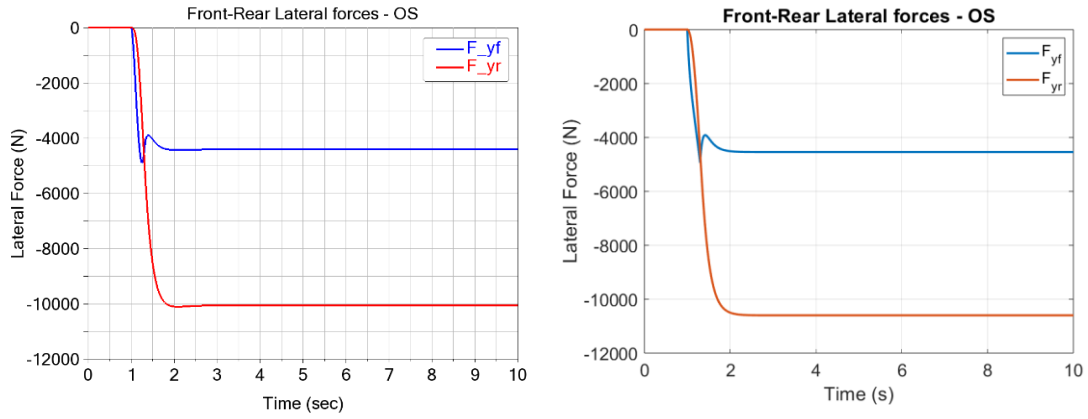


Figure 10: Oversteer vehicle results.

The effect of slip angles relationship on vehicle's path is clearly visible in Figure 11, which compares vehicle trajectory and yaw rate for the three steering conditions. Oversteer vehicle experiences the lowest turn radius, understeer the greatest and neutral-steer lies between them. This behaviour is reflected by yaw rate, which becomes higher for lower turning radius. The transient response is characterized by negligible overshoot, indicating good handling performance.

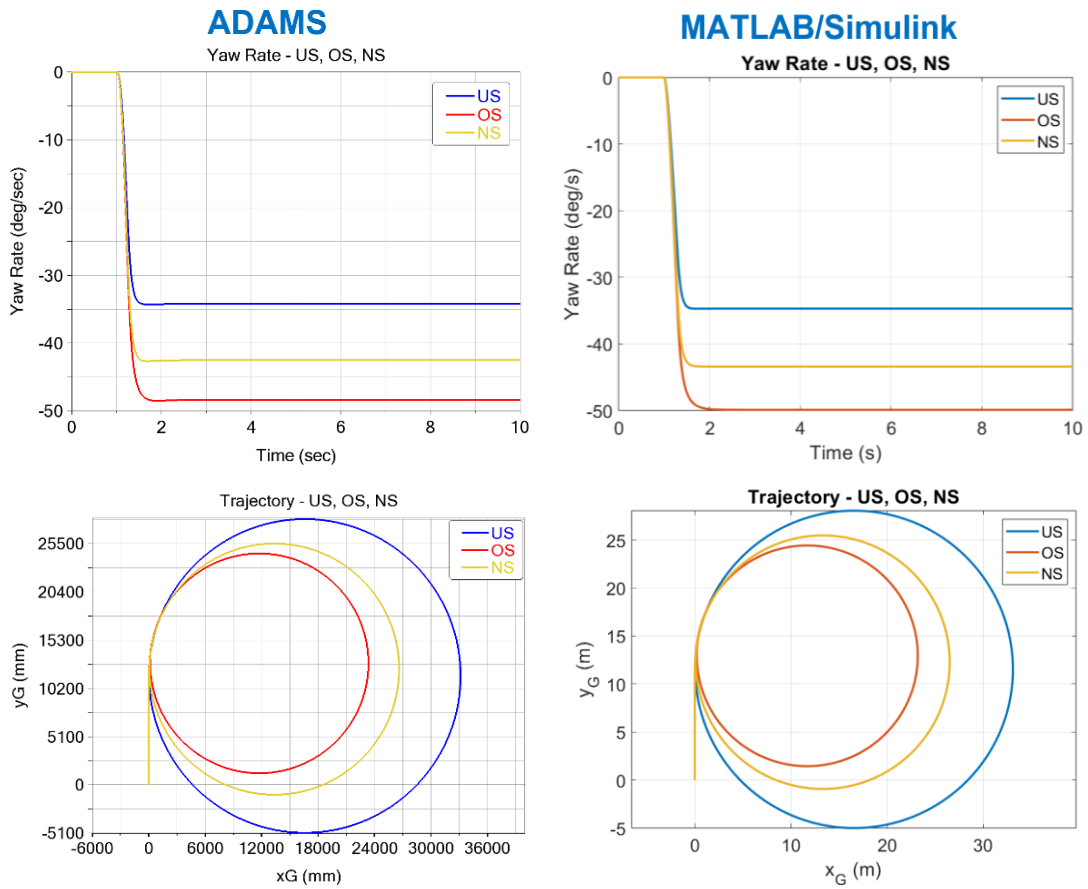


Figure 11: US, OS, NS trajectory comparison.

Vehicle steering behaviour can also be assessed through the understeer gradient and the curvature gain, evaluated as (Genta and Morello, 2020):

$$K = \frac{m}{l} \left(\frac{b}{C_1} - \frac{a}{C_2} \right) \quad (8)$$

$$\frac{\rho_{ss}}{\delta} = \frac{1}{l + KV^2} \quad (9)$$

- If $K = 0$, ρ_{ss}/δ is constant and the vehicle is neutral-steer.
- If $K > 0$, ρ_{ss}/δ decreases with increasing speed and the vehicle is understeer.
- If $K < 0$, ρ_{ss}/δ increases with increasing speed until critical speed, beyond which the vehicle becomes unstable and requires counter-steering, typical of oversteer (Genta and Morello, 2020).

Figure 12 presents the curvature gain obtained from the developed models, while Table 1 reports the understeer gradients. The obtained results confirm the expected understeer, oversteer and neutral-steer behaviours, further validating the models.

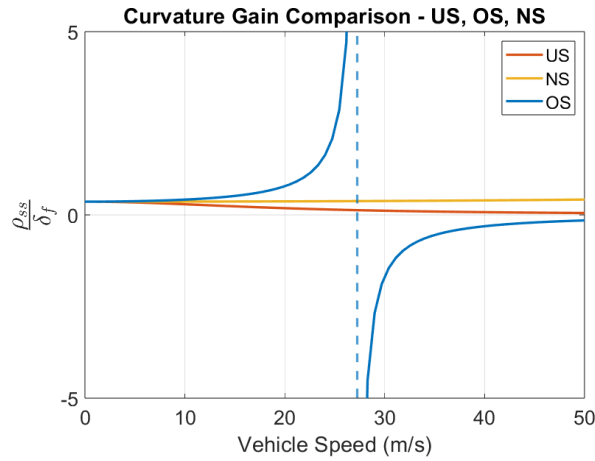


Figure 12: Curvature gain at different vehicle speed and configurations.

Understeering Gradient		
US	OS	NS
0.0068	-0.0038	6.5e-5

Table 1: Understeering gradient for understeer, oversteer, neutral-steer vehicle.

The linear model in this simulation shows that large side-slip angles are developed; therefore, it is not reliable to assess vehicle's lateral behaviour as slip angles are beyond linear range (Milliken and Milliken, 1995).

2.2.2. Nonlinear

This section analyses the effect of nonlinear model at $V_x = 30 \text{ m/s}$, a value of speed close real-world standards (ISO, 2011). As shown in Figure 13, accounting for tyre nonlinearity leads to a significant reduction in yaw rate, driven by the lower lateral forces the tyres can produce. The large gap between linear and nonlinear results indicates that tyres operate at high slip angles, well outside the spline's linear region; if slip angles had remained small, the two responses would be much similar. These differences are also evident in the vehicle path, which shows a much larger curvature radius in the nonlinear case. The obtained results indicate a more realistic simulation, as the yaw-rate

assumes values closer to those observed in real vehicle operation (Asperti *et al.*, 2025).

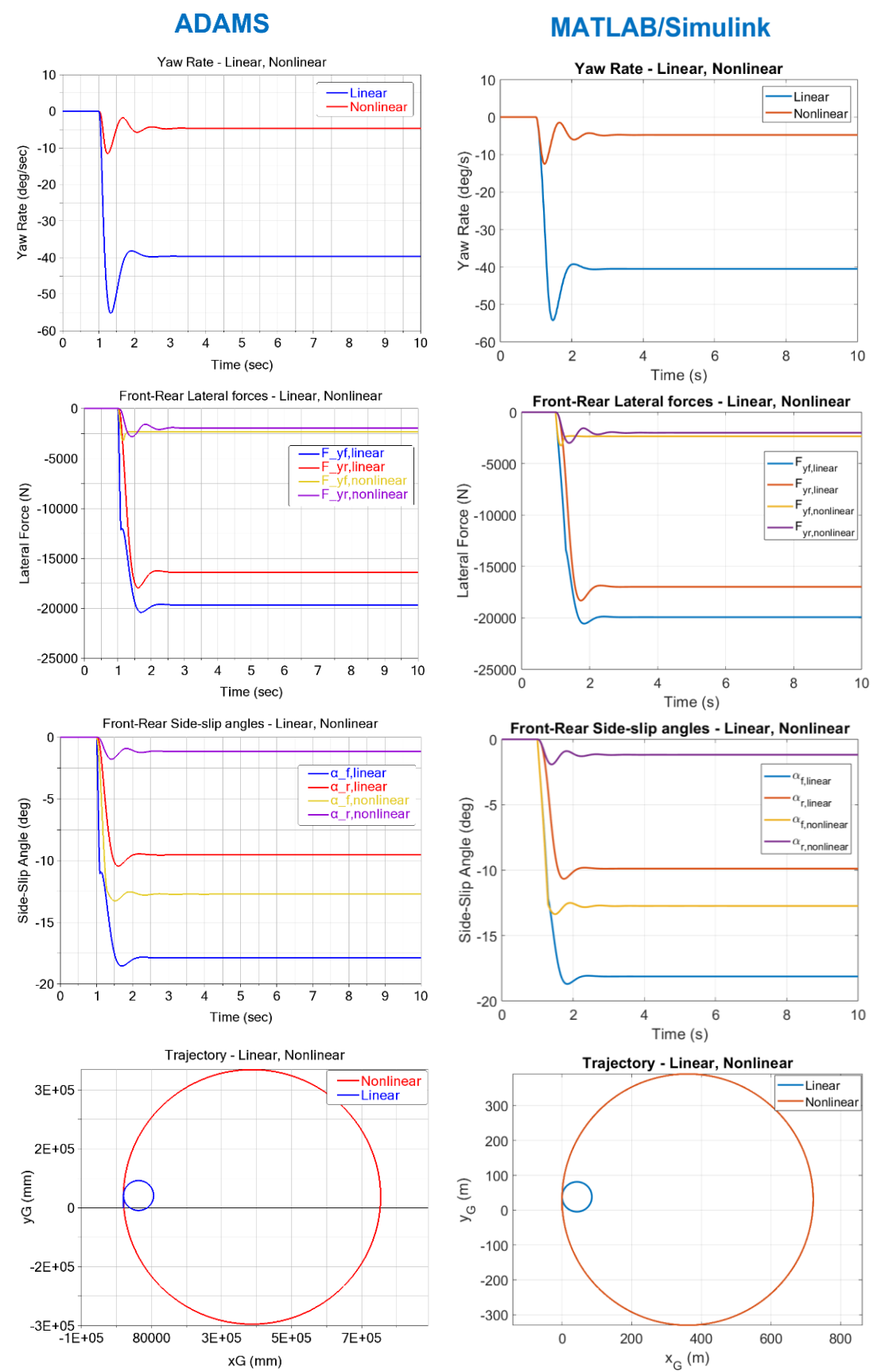


Figure 13: Nonlinear vehicle results.

2.2.3. Transient

Including tyre dynamics in the lateral-force formulation introduces a delay in force build-up compared to the case without transient effects. As shown in Figure 14, which provides a zoomed-in view of the lateral-force curves, the model with transients exhibits a delay when the system leaves steady-state conditions, i.e., during the manoeuvre's transient phase. In steady state, both before and after the step input, the curves overlap perfectly. This behaviour arises from the analytical formulation of the adopted model, which incorporates a derivative term to capture transient effects, as indicated in Equation 4.

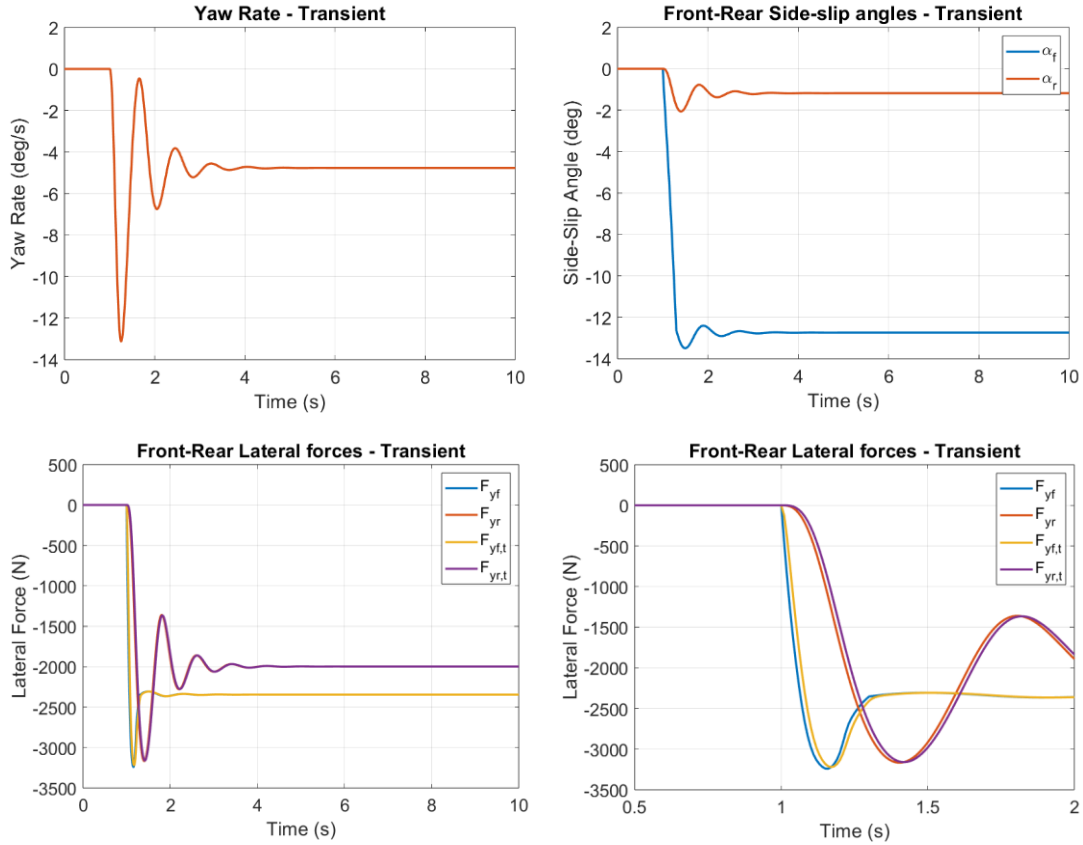


Figure 14: Transient model results.

2.2.4. Double-Track

As shown in Figure 15, the double-track model follows a wider trajectory due to a lower yaw-rate value, indicating reduced lateral force generation. This is a direct consequence of the nonlinear cornering stiffness-vertical load relationship: load transfer raises the outer tyres' stiffness and lowers the inner ones, but the variations are not proportional. Therefore, the resulting net axle stiffness of the double-track model is lower than in the single-track model. This condition is highlighted by Figures 16, which illustrate the operating points on the stiffness-load curve, and in Table 2, reporting the corresponding front and rear axle stiffness values for both models. The vehicle's handling response can be evaluated by examining its transient behaviour from the yaw-rate chart. In

both cases, the vehicle exhibits a pronounced overshoot of approximately 230%, indicating poor performance in handling rapid steering inputs.

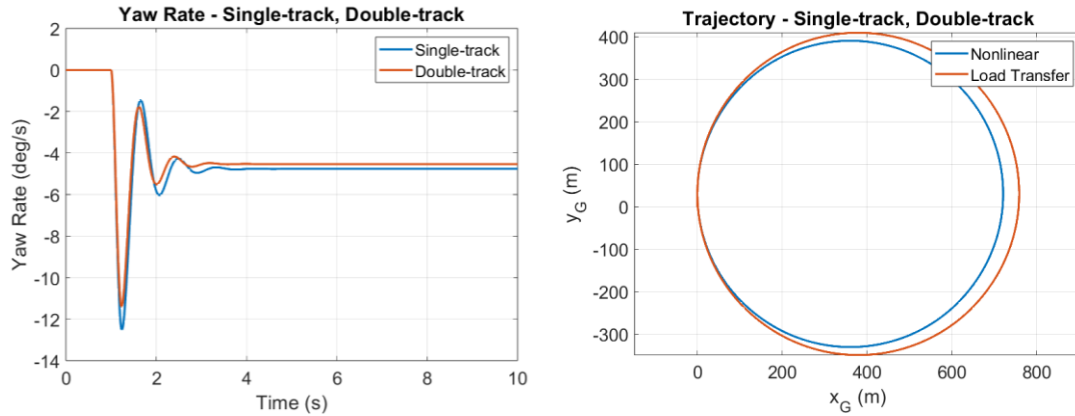


Figure 15: Linear and nonlinear trajectory and yaw-rate comparison.

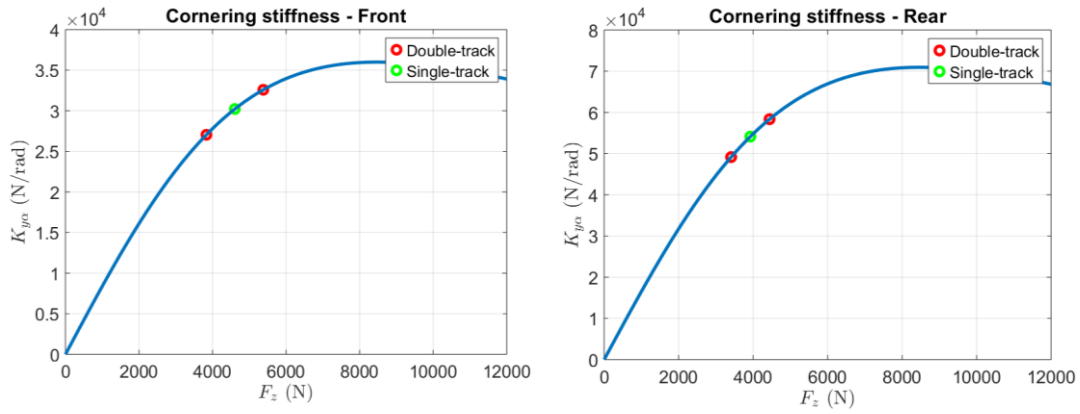


Figure 16: Front-Rear cornering stiffness curve with single and double-track operating points.

	Single-track	Double-track
C_f (N/deg)	1054.6	1040.5
C_r (N/deg)	1889.2	1876.0

Table 2: Front-Rear cornering stiffness for single and double-track models.

2.2.5 Derivative Analysis

Vehicle cornering response can also be assessed analytically using derivatives of stability. Refer to Appendix G for more details.

3. Suspension dynamics and ride optimisation

3.1. Description and Discussion of Models

3.1.1 Quarter-car Model

The quarter-car-model represents a simplified version of a vehicle's suspension system. In the automotive field, it is widely used during the early stages of suspension design because it enables engineers to study how spring, damping and tyre characteristics affect ride comfort, handling and safety, while keeping development costs low, as its implementation on a test rig is significantly cheaper than that of more complex models (Kumar, Pasupuleti and Natarajan, 2023). Its broad applicability also stems from the fact that it can be analysed in both frequency and time domains, allowing to evaluate how suspensions parameters influence the vertical response of the wheel hub and chassis under various excitation frequencies.

Despite its simplicity, the quarter-car model can effectively represent ride dynamics (Gurley, 2014). However, some limitations are present. Since it accounts only for vertical motion, it performs well when vehicle inputs lie in a single plane, making it more suitable for low-speed maneuvers where roll and yaw motions are limited. Additionally, the model neglects chassis rotational dynamics, meaning that all the energy transmitted to the chassis is converted exclusively into translational motion. As a result, the quarter-car model tends to overestimate the vehicle's response (Belfiore *et al.*, 1993). Details about models' implementation are available in Appendix E.

3.1.1.1 Single Degree of Freedom

The single-degree-of-freedom quarter-car model treats tyres as rigid and massless bodies, considering only the sprung mass. This model provides reliable representation of motion only at low excitation frequency, specifically within the natural frequency range of sprung mass, i.e. 3-5 Hz (Genta and Morello, 2020). A simple representation of the model is provided in Figure 17, while the equation of motion characterizing the model is presented in Equation 10.

$$m\ddot{x} + c(\dot{x} - \dot{y}) + k(x - y) = 0 \quad (10)$$

The road excitation is represented by the variable y , which is imposed as a step of 25 mm to simulate a bump, while x represents the degree-of-freedom of the model. For a simpler analysis, the damping coefficient is considered constant, even though damping is typically the main parameter optimised in suspension tuning. In this context, the damping ratio is used, defined as:

$$\zeta = \frac{c}{C_c} \quad (11)$$

where the critical damping C_c is:

$$C_c = 2\sqrt{km} \quad (12)$$

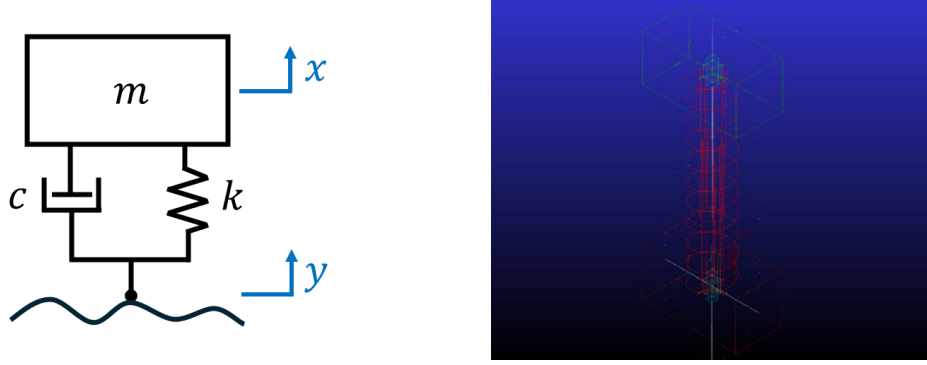


Figure 17: 1-DOF quarter-car model.

3.1.1.2 Two Degrees of Freedom

Neglecting tyre mass and stiffness limits the accuracy of the 1-DOF model. A more realistic representation is the 2-DOFs model, which represents tyres as massless spring with negligible damping and accounts for both sprung and unsprung masses. This model provides accurate estimation of vertical motion for frequencies up to the natural frequency of the unsprung mass, typically in the range 30-50 Hz (Genta and Morello, 2020).

The underlying equations of motion describing the 2-DOFs quarter-car model are reported in Equation 13. Two variants are implemented: one with a constant damping coefficient and one with a nonlinear damper. An optimisation strategy is also implemented to improve both comfort and handling.

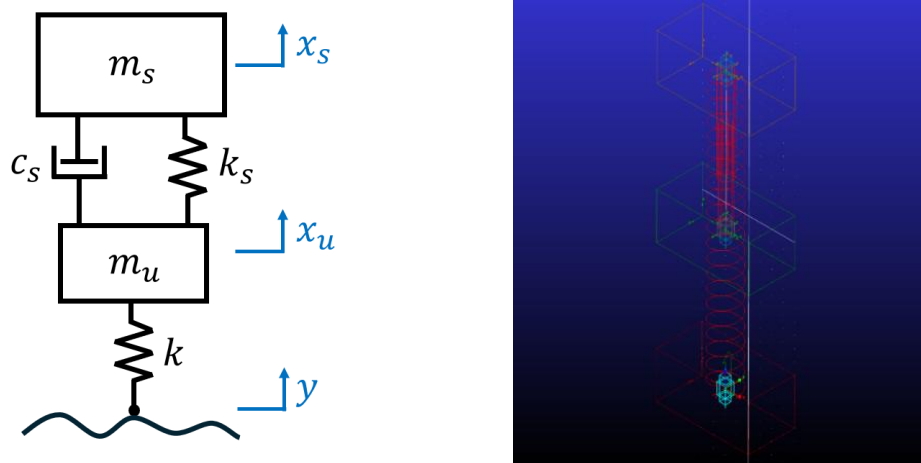


Figure 18: 2-DOFs quarter-car model.

$$\begin{cases} m_s \ddot{x}_s + c_s(\dot{x}_s - \dot{x}_u) + k_s(x_s - x_u) = 0 \\ m_u \ddot{x}_u + c_s(\dot{x}_u - \dot{x}_s) + k_s(x_u - x_s) + k_t(x_u - y) = 0 \end{cases} \quad (13)$$

3.1.2 Half-car Model

The simplified mechanics of the quarter-car model do not fully capture the rigid-body motions that may occur in a real vehicle. Because of the longitudinal distance between the axles, a vehicle behaves as a multi-input system that responds also to vertical bounce and pitch motion. Understanding these coupled motions is essential because their interactions determine the vertical vibrations at any point on the vehicle, directly impacting on ride comfort (Gillespie, 2021).

For this reason, a 4-DOFs half-car model is developed to incorporate also pitch and bounce motions to provide a more realistic representation of the vehicle's vertical dynamics. This model captures pitch, bounce and vertical motions of front and rear unsprung masses (Debnath *et al.*, 2022), as shown by Figure 19.

The four equations of motion describing the half-car model are reported in Equation 14. Unlike quarter-car model, the half-car model features two inputs: one at the front and one at the rear. For a realistic simulation, only the front input is directly imposed, while the rear input is equal to the front one but delayed by a quantity proportional to the wheelbase divided by vehicle speed, which in the simulation was set to 10 m/s.

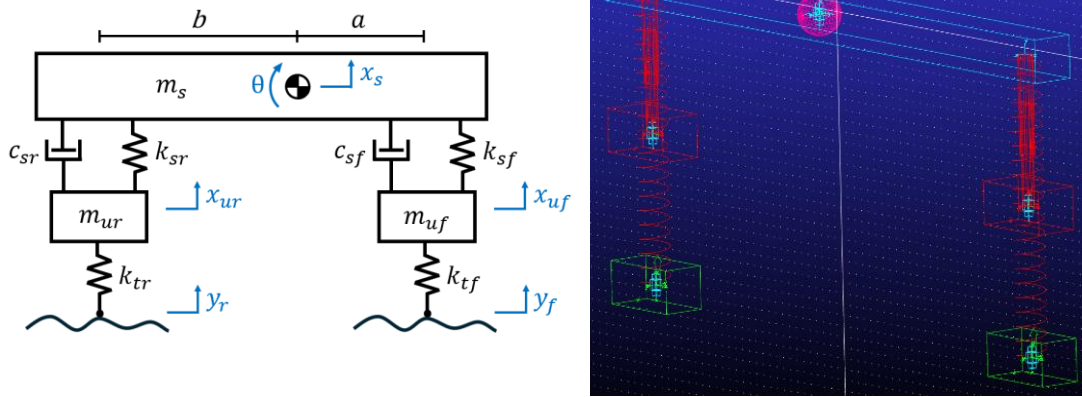


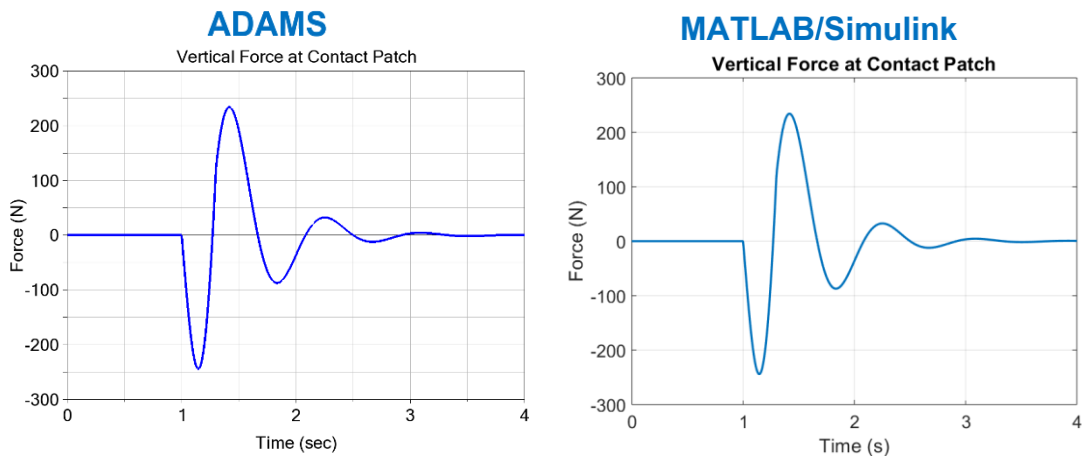
Figure 19: 4-DOFs half-car model.

$$\begin{cases} m_s \ddot{x}_s + c_{sf}(\dot{x}_s - \dot{x}_{uf} - a\dot{\theta}) + c_{sr}(\dot{x}_s - \dot{x}_{ur} + b\dot{\theta}) + k_{sf}(x_s - x_{uf} - a\theta) + k_{sr}(x_s - x_{ur} + b\theta) = 0 \\ I_y \ddot{\theta} - ac_{sf}(\dot{x}_s - \dot{x}_{uf} - a\dot{\theta}) + bc_{sr}(\dot{x}_s - \dot{x}_{ur} + b\dot{\theta}) - ak_{sf}(x_s - x_{uf} - a\theta) + bk_{sr}(x_s - x_{ur} + b\theta) = 0 \\ m_{uf} \ddot{x}_{uf} - c_{sf}(\dot{x}_s - \dot{x}_{uf} - a\dot{\theta}) - k_{sf}(x_s - x_{uf} - a\theta) + k_{tf}(x_{uf} - y_f) = 0 \\ m_{ur} \ddot{x}_{ur} - c_{sr}(\dot{x}_s - \dot{x}_{ur} + b\dot{\theta}) - k_{sr}(x_s - x_{ur} + b\theta) + k_{tr}(x_{ur} - y_r) = 0 \end{cases} \quad (14)$$

3.2 Results & Discussion

3.2.1 Single Degree of Freedom

This section presents the ADAMS and MATLAB/Simulink bump-response results, focusing on sprung mass acceleration and tyre vertical force at the contact patch, which are key indicators for comfort and handling.



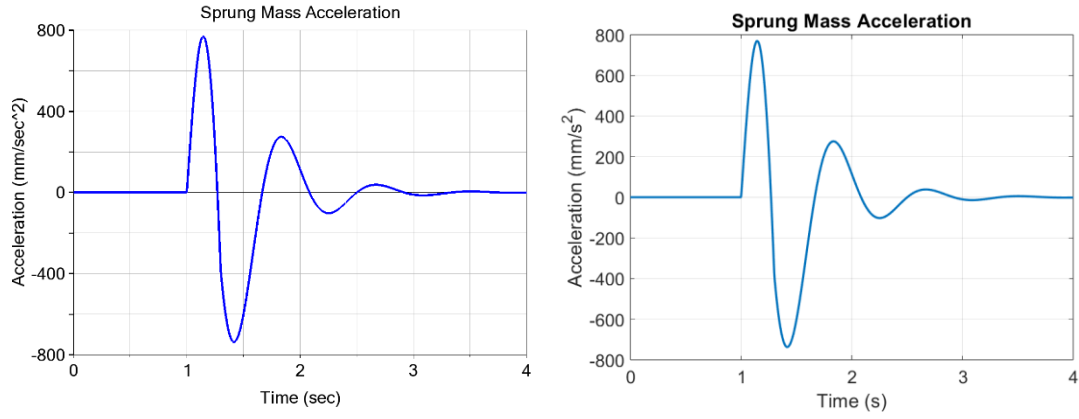


Figure 20: 1-DOF results.

Suspension behaviour is assessed through three KPIs: two related to handling, the Dynamic Load Coefficient (DLC) and the Dynamic Load Factor (DLF), representing respectively the RMS and peak-to-peak variations of vertical tyre load relative to the static value, and one related to comfort, defined as the RMS of chassis' vertical acceleration. Reference thresholds taken from existing standards or technical publications are reported in Figure 21, while Table 3 shows the KPI values obtained from the simulations. These results indicate that the suspension achieves a satisfactory behaviour for both comfort and handling, as all the KPIs do not exceed the reference thresholds.

$a_v \leq 0.315 \text{ m/s}^2$	Not uncomfortable	<table><tr><th>DLC</th><th>DLF</th></tr><tr><td>0.1÷0.15</td><td>0.3÷0.4</td></tr></table>	DLC	DLF	0.1÷0.15	0.3÷0.4
DLC	DLF					
0.1÷0.15	0.3÷0.4					
$0.315 \text{ m/s}^2 \leq a_v \leq 0.63 \text{ m/s}^2$	A little uncomfortable					
$0.5 \text{ m/s}^2 \leq a_v \leq 1 \text{ m/s}^2$	Fairly uncomfortable					
$0.8 \text{ m/s}^2 \leq a_v \leq 1.6 \text{ m/s}^2$	Uncomfortable					
$1.25 \text{ m/s}^2 \leq a_v \leq 2.5 \text{ m/s}^2$	Very uncomfortable					
$a_v \geq 2 \text{ m/s}^2$	Extremely uncomfortable					

Figure 21: Comfort and handling KPIs reference values (Amati, 2025), (Cebon and Winkler, 1991), (Rashid, Lee and Dawkins, 1972).

ADAMS			MATLAB/Simulink		
$\ddot{x}_{s,rms}$	DLC	DLF	$\ddot{x}_{s,rms}$	DLC	DLF
0.2276	0.232	0.1537	0.2278	0.0232	0.1536

Table 3: 1-DOF quarter-car model KPI.

A frequency-domain analysis using Bode diagrams and state-space representation further supports this assessment (see Appendix I for details about implementation). Figure 22 shows the frequency response of sprung-mass acceleration for the 1-DOF quarter-car model, computed for several damping ratios, as well as for the real system, represented by the highlighted curve. The damping ratios are generated using:

$$c = 2m\omega_n\zeta \quad (15)$$

$$\omega_n = \sqrt{\frac{k}{m}} \quad (16)$$

where the natural frequency is 7.885 rad/s . The real system response closely matches the curve corresponding to a damping ratio of about 0.3. This value represents a good design compromise: it avoids excessive amplification around resonance while keeping acceleration low across the frequency range. Higher damping reduces low-frequency acceleration but worsens the post-resonance response, whereas lower damping has the opposite effect, penalising comfort at low frequencies.

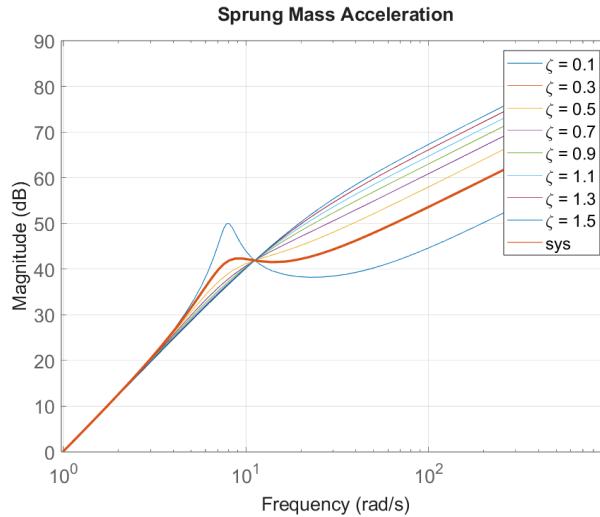


Figure 22: Sprung mass acceleration frequency response.

3.2.2 Two Degrees of Freedom

3.2.2.1 Linear Damping

The frequency response of the 2-DOF quarter-car model is shown in Figure 23. With the introduction of the unsprung mass and the second degree-of-freedom, the system exhibits two natural frequencies, whose values are $\omega_1 = 1.25 \text{ Hz}$ and $\omega_2 = 10.9 \text{ Hz}$ (see Appendix F for details). As in the 1-DOF case, with the suspension damping ratio of the reference vehicle, the frequency response remains relatively low across the entire frequency range for both sprung-mass acceleration and tyre vertical force.

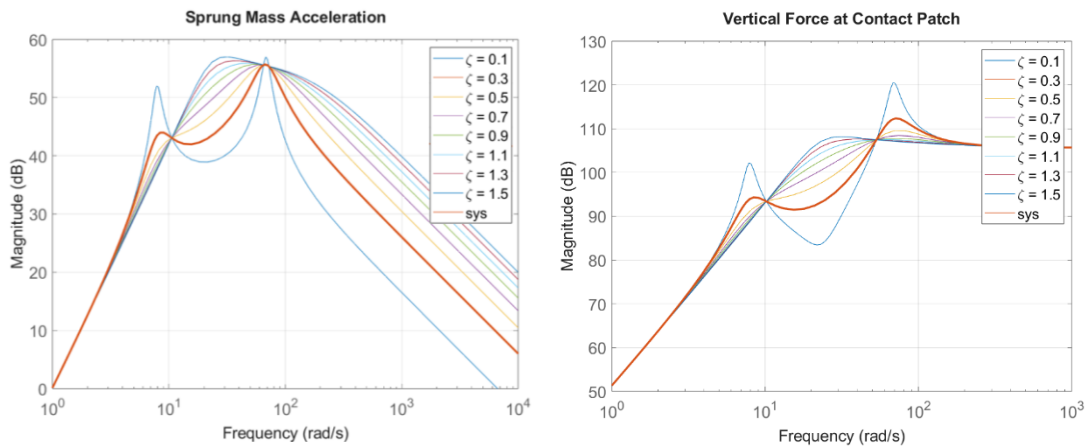


Figure 23: Sprung mass acceleration and vertical load at tyre contact patch frequency response.

3.2.2.2 Nonlinear Damping

Although the frequency response analysis suggests that the selected damping value provides a good compromise between comfort and handling, an optimisation process was carried out to determine the best combination of damping coefficient and spring stiffness. The objective is to optimise comfort, by minimizing vertical chassis acceleration, and handling, by minimizing the variation of vertical load at the contact patch. To increase the model fidelity to modern passenger cars, a nonlinear damper was also implemented. Its force-velocity characteristic, measured using a dyno test rig, is shown in Figure 24.

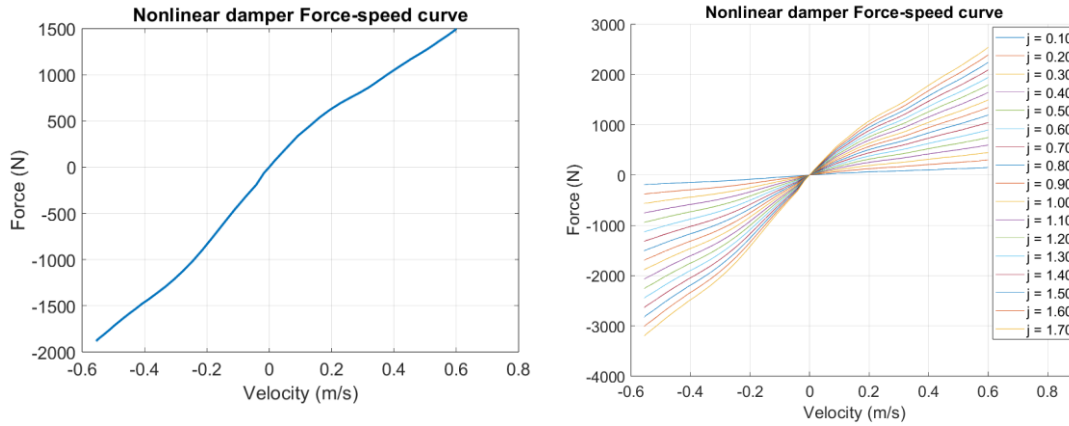


Figure 24: Real (left) and rescaled (right) nonlinear damper characteristic curve.

For the optimisation, parameter ranges for both stiffness and damping must be defined. The stiffness range was selected so that the first natural frequency, associated with the sprung mass, spans 1.15-1.35 Hz, a typical design choice to avoid resonance within the highly sensitive human frequency range (4-8 Hz) and below the motion-sickness range (<1 Hz) (Tonoli, 2024). Because a nonlinear damper is used, its optimisation is performed on a scaling factor j , ranging from 0.1 to 1.7, that modifies the slope of the force-velocity curve, thereby changing the damping level. The effect of j on the damper characteristic is illustrated in Figure 24.

The optimisation maps produced in MATLAB/Simulink, showing the RMS variation of vertical chassis acceleration and tyre load, are shown in Figure 25.

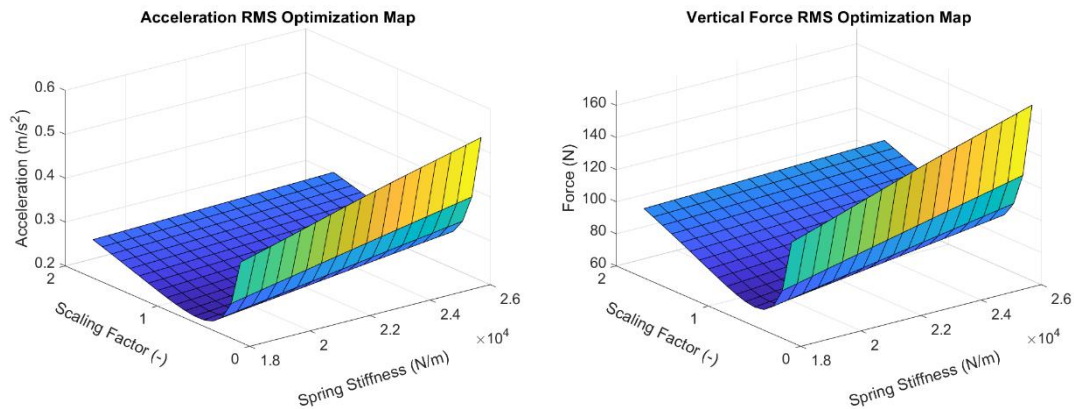


Figure 25: Optimization maps – MATLAB/Simulink.

From these maps, the combinations of stiffness and damping offering the best performance can be identified. For handling, the minimum tyre-force variation is achieved with $k = 18 \text{ kN/m}$ and $j = 0.5$. For comfort, the minimum vertical acceleration is obtained with $k = 18 \text{ kN/m}$ and $j = 0.6$. The corresponding nonlinear damper curves for the optimised configurations are shown in Figure 27. The ADAMS results, reported in Figure 26, and obtained through an optimisation based only on tyre force, yield the same spring stiffness and scaling factor values, thereby cross-validating the optimisation procedures.

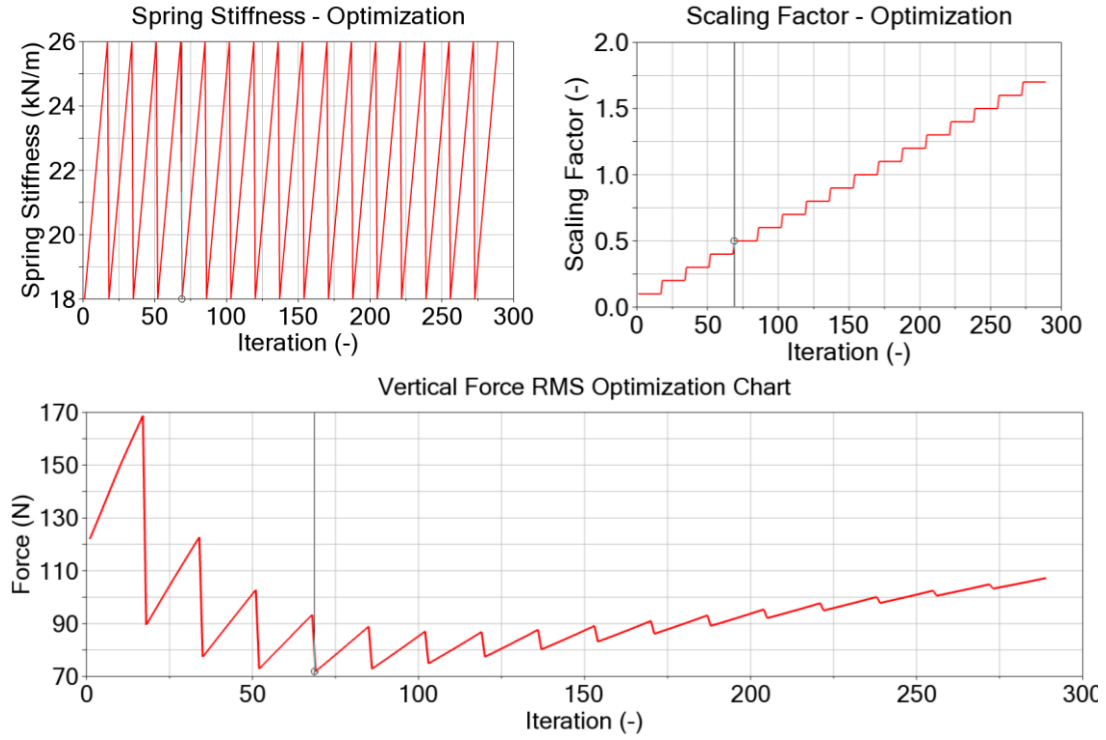


Figure 26: Optimization results - ADAMS.

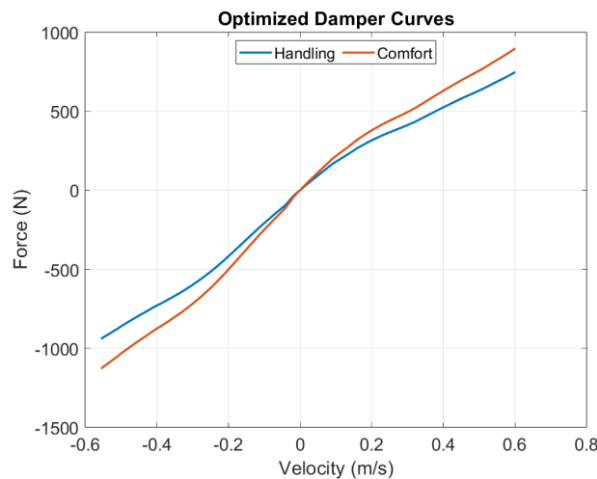


Figure 27: Optimized damper curves.

To obtain a balanced trade-off between comfort and handling, a scaling factor of $j = 0.55$ is selected as the final design parameter. The optimised

suspension model is then compared with the baseline configuration. The optimised setup improves both comfort and handling performance compared to the original vehicle, as KPIs, reported in Table 4, and peak values, visible in Figure 28, are lower.

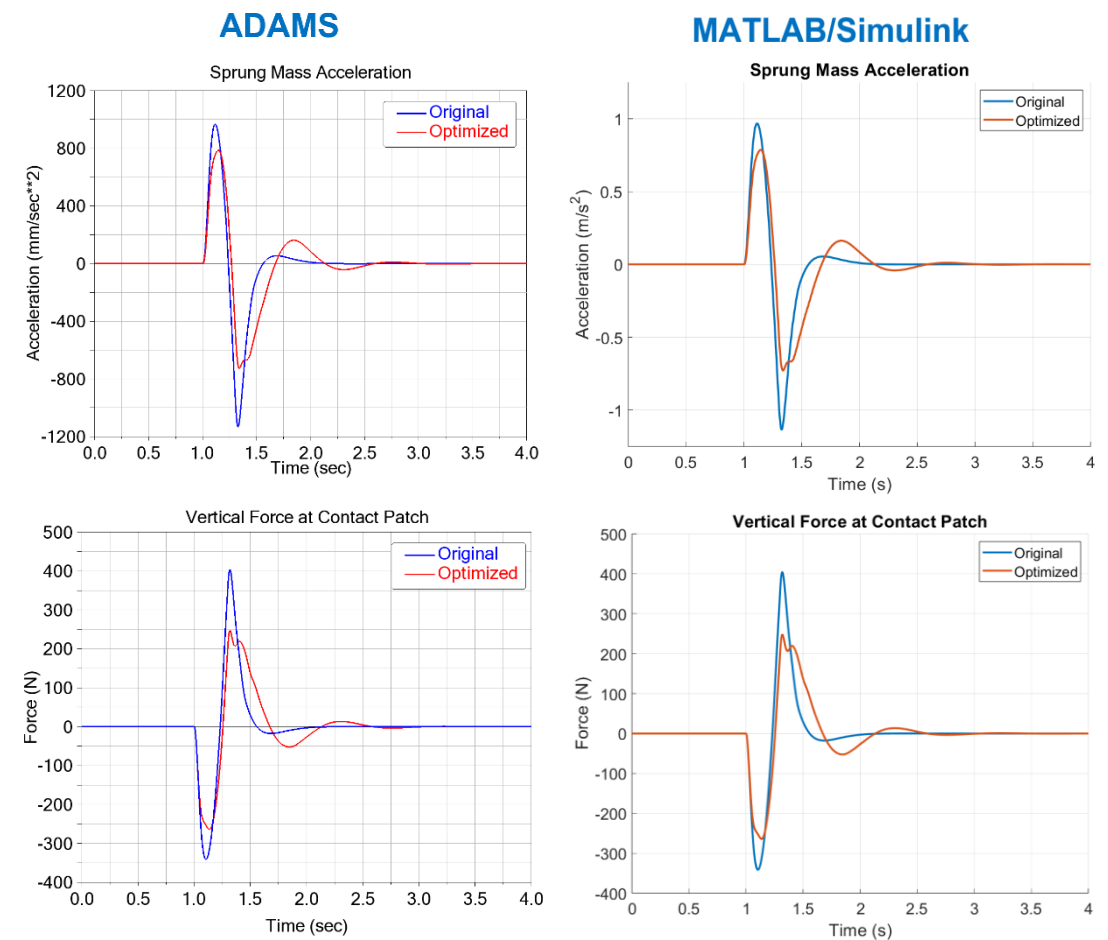


Figure 28: Optimized and original bump response.

ADAMS			
Comfort		Handling	
$\ddot{x}_{s,rms}$		DLC	DLF
Original	Optimized	Original	Optimized
0.236	0.214	0.027	0.165

MATLAB/Simulink			
Comfort		Handling	
$\ddot{x}_{s,rms}$		DLC	DLF
Original	Optimized	Original	Optimized
0.236	0.214	0.027	0.164

Table 4: 2-DOFs quarter-car model handling and comfort KPIs comparison.

3.2.3 Half-Car Model

The vertical and pitch acceleration frequency responses, reported in Figure 29, exhibit three resonances: two low-frequency peaks around 1.1 Hz, associated with heave and pitch and consistent with the primary resonance observed in the quarter-car model, and one high-frequency peak from the unsprung masses. Although two separate unsprung-mass resonances would normally appear, the similar front and rear unsprung masses and stiffnesses in the studied vehicle cause them to merge into a single high-frequency peak.

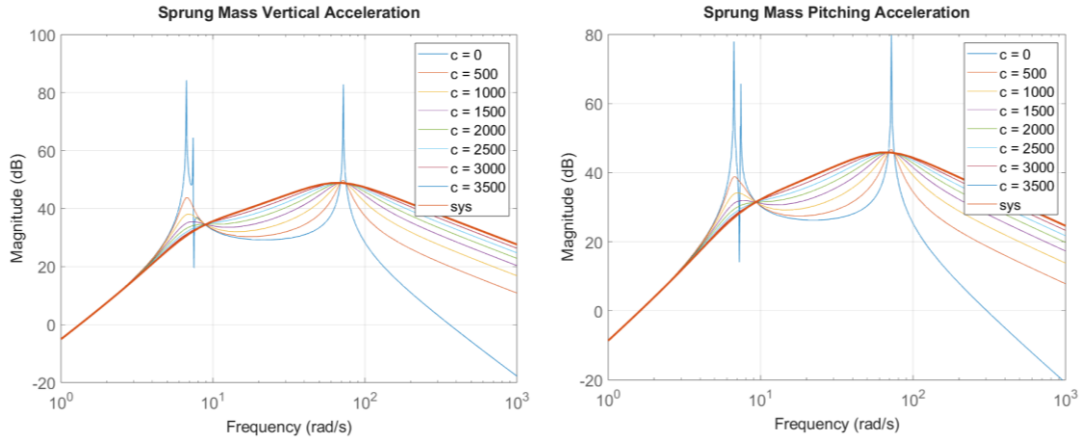


Figure 29: Vertical and pitching acceleration frequency response.

The half-car model is simulated using the suspension parameters optimised with the quarter-car model, together with a realistic road input based on the ISO 8606 Class B profile (ISO, 1995) and the human sensitivity weighting from ISO 2631 (ISO, 1997) (see Appendix E for details). A schematic flowchart illustrating the logic is shown in Figure 30.

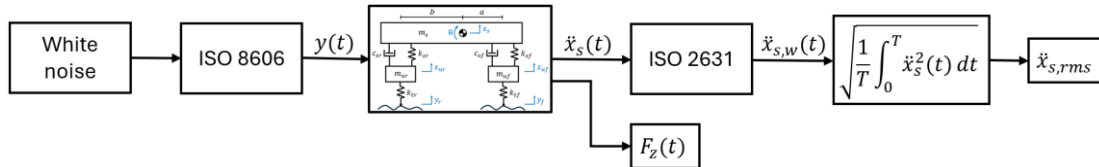


Figure 30: Flowchart to evaluate half-car model response with real road input.

The results, reported in Figure 31 and Table 5, indicate that the optimised suspension improves both comfort and handling even in this more realistic model, confirming the quarter-car model's optimisation effectiveness for evaluating vertical vehicle dynamics.

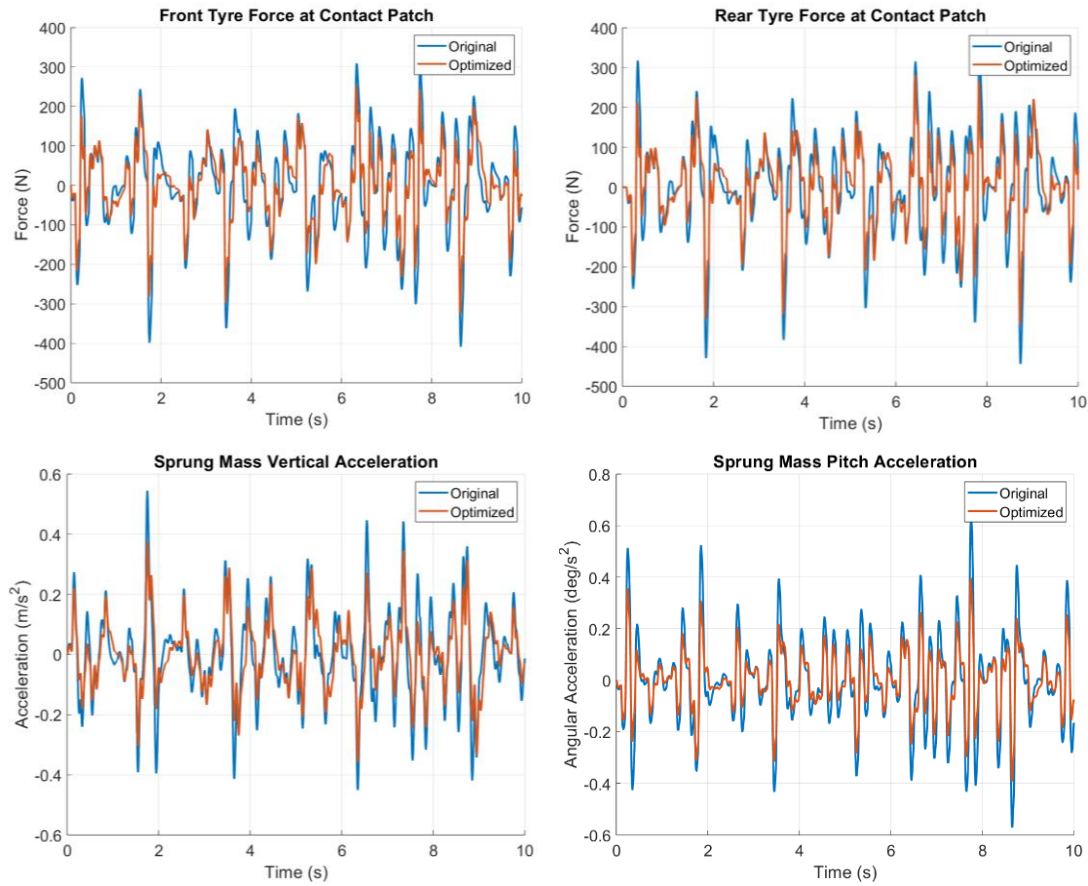


Figure 31: Half-car model results.

Comfort			
$\ddot{x}_{s,rms}$		$\ddot{\theta}_{s,rms}$	
Original	Optimized	Optimized	Original
0.149	0.112	0.176	0.114

Handling				
	DLC		DLF	
	Original	Optimized	Original	Optimized
Front	0.029	0.021	0.175	0.141
Rear	0.036	0.026	0.225	0.178

Table 5: 4-DOFs half-car model handling and comfort KPIs comparison.

4 Individual Reflection

Although the most complex models analysed provides good estimations of vehicle performance, several enhancements can be implemented. More complex models could be adopted, incorporating body roll, tyres camber variation and accounting for flexibility of chassis to improve fidelity. Moreover, adding tyre characteristic envelope would further enhance accuracy, since vehicle's response is highly affected by tyre behaviour (Debnath *et al.*, 2022). Regarding vertical motion analysis, a study focused on vehicle's response to

different operating speed could be carried out, since speed strongly impacts on suspension behaviour, and therefore on both handling and passengers' comfort (Genta and Morello, 2020).

This work allowed me to deal with a wide range of vehicle dynamics models, understanding the limitations of each and learning how to improve their accuracy step-by-step. Working with ADAMS also allowed me to develop fundamental skills with a tool that is essential for future work in the field of vehicle dynamics. Moreover, the critical analysis conducted to understand the physical motivations behind the simulation results, supported by an extensive reading of technical publications, textbooks and the theoretical concepts learned during lectures, strengthen my knowledge of fundamental vehicle dynamics principles, providing a solid foundation for further studies and applications related to this topic.

5 References

Amati, N. (2025) *Driver Assistance System Design - Part B*. 01USHWM. Lecture notes. Politecnico di Torino. Department of Mechanical and Aerospace Engineering.

Asperti, M. *et al.* (2025) 'Multi Input Multi Output Lateral Dynamics Control of an Electric Vehicle', *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, 239(8), pp. 3322–3333. Available at: <https://doi.org/10.1177/09544070241252197>.

Belfiore, D. *et al.* (1993) 'Vehicle Axle Accelerations Due to Road Roughness for Accelerated Life Testing', *SAE Technical Paper 930256*. Available at <https://doi.org/10.4271/930256>.

Crolla, D.A. (1991) *An Introduction to Vehicle Dynamics*. University of Leeds, Department of Mechanical Engineering.

Debnath, S. *et al.* (2022) 'Two Wheeler Suspension Tuning with Extended Half-Car Model', *SAE Technical Paper 2022-28-0087*. Available at: <https://doi.org/10.4271/2022-28-0087>.

Genta, G. and Morello, L. (2020) *The Automotive Chassis Volume 2: System Design*. 2nd edn. Springer.

Gillespie, T.D. (2021) *Fundamentals of Vehicle Dynamics*. 2nd edn. SAE International.

Gurley, A. (2014) 'The Rolling Quarter Car Model a Method to Incorporate Dynamic Tire Response in Grip Optimization', *SAE Technical Paper 2014-01-0156*. Available at: <https://doi.org/10.4271/2014-01-0156>.

ISO (1997) 2631: *Mechanical vibration and shock - Evaluation of human exposure to whole-body vibration*. Geneva: International Organization for Standardization.

ISO (2011) 7401: *Road vehicles - Lateral transient response test methods*. Geneva: International Organization for Standardization.

ISO (1995) 8606: *Mechanical vibration - Road surface profiles - Reporting of measured data*. Geneva: International Organization for Standardization.

K. Jiang, A. C. Victorino and A. Charara (2016), 'Real-Time Estimation of Vehicle's Lateral Dynamics at Inclined Road Employing Extended Kalman Filter', *IEEE 11th Conference on Industrial Electronics and Applications (ICIEA)*, pp. 2360-2365, doi: 10.1109/ICIEA.2016.7603987.

Kim, J. (2025) *Autodrive: Vehicle Dynamics and Bicycle Model*. Available at: https://gaussian37.github.io/autodrive-ose-vehicle_dynamics_and_bicycle_model/ (Accessed: 16 December 2025).

Kumar, V.S., Pasupuleti, T. and Natarajan, M. (2023) 'Simulation and Analysis of Quarter Car Model for Low Cost Suspension Test Rig', *SAE Technical Paper 2023-28-0164*. Available at: <https://doi.org/10.4271/2023-28-0164>.

Loeb, J.S. *et al.* (1990) 'Lateral Stiffness, Cornering Stiffness and Relaxation Length of the Pneumatic Tire', *SAE Technical Paper 900129*. Available at: <https://doi.org/10.4271/900129>.

MacInnis, D.D. *et al.* (1997) 'A Comparison of Moment of Inertia Estimation Techniques for Vehicle Dynamics Simulation', *SAE Technical Paper 970951*. Available at: <https://doi.org/10.4271/970951>.

Milliken, W.F. and Milliken, D.L. (1995) *Race Car Vehicle Dynamics*. 1st edn. SAE International.

Mitsunori, T. (2021) 'A Clarification of Relationship Steering Force and Yaw Characteristics to Line Traceability in Small Steering Angle', *Nissan Technical Review*. Available at: <https://www.nissan-global.com/EN/TECHNICALREVIEW/>.

Nissan Motor Corporation (2017) *Nissan GT-R*. Available at: <https://autocatalogarchive.com/wp-content/uploads/2018/12/Nissan-GT-R-Nismo-2017-AU.pdf>.

Novara, C. (2025) *Driver Assistance System Design - Part A*. 01USHWM. Lecture notes. Politecnico di Torino. Department of Mechanical and Aerospace Engineering.

Polack, P. *et al.* (2017) 'The Kinematic Bicycle Model: a Consistent Model for Planning Feasible Trajectories for Autonomous Vehicles', *IEEE Intelligent Vehicles Symposium (IV)*, pp. 812-818, doi: 10.1109/IVS.2017.7995816.

Sondkar, P. *et al.* (2025) 'Coupled Lateral-Longitudinal Vehicle Dynamics Modeling to Predict the Loads Experienced by Side Gears in the Differentials', *SAE Technical Paper 2025-01-8519*. Available at: <https://doi.org/10.4271/2025-01-8519>.

Talukdar, S. and Kulkarni, S. (2011) 'A Comparative Analysis of a Rigid Bicycle Model with an Elastic Bicycle Model for Small Trucks', *SAE Technical Paper 2011-01-0240*. Available at: <https://doi.org/10.4271/2011-01-0240>.

Tonoli, A. (2023) *Motor Vehicle Design*. 04LNLXP. Lecture notes. Politecnico di Torino. Department of Mechanical and Aerospace Engineering.

Tonoli, A. (2024) *Car Body Design and Aerodynamics*. 03OFGWM. Lecture notes. Politecnico di Torino. Department of Mechanical and Aerospace Engineering.

Words count (excluding cover page, reference list, appendices and individual reflection) = 3298

Appendix A

This section presents the steady-state numerical results obtained from the different simulations. The charts reported in the main sections compare the time evolution of the different parameters and are useful to compare the transient behaviour of the vehicle. The numerical steady-state values reported in this appendix can instead be used to assess the accuracy of the simulations and to cross-validate ADAMS and MATLAB/Simulink models.

Lateral Dynamics

Linear

LINEAR						
	US		OS		NS	
	ADAMS	MATLAB	ADAMS	MATLAB	ADAMS	MATLAB
$\dot{\psi}$ (deg/s)	-34.2	-34.7	-48.3	-49.9	-42.5	-43.4
α_f (deg)	-5.17	-5.17	-4.52	-4.65	-4.65	-4.75
α_r (deg)	-2.76	-2.82	-6.19	-6.52	-4.69	-4.81
F_{yf} (N)	-5678	-5688	-4411	-4544	-5006	-5072
F_{yr} (N)	-4732	-4845	-10056	-10602	-7818	-8102

Table 6: Steady-state results from ADAMS and MATLAB/Simulink for the linear single-track model under understeer, oversteer, and neutral-steer conditions.

Nonlinear

NONLINEAR		
	ADAMS	Simulink
$\dot{\psi}$ (deg/s)	-4.68	-4.77
α_f (deg)	-12.7	-12.7
α_r (deg)	-1.15	-1.18
F_{yf} (N)	-2352	-2345
F_{yr} (N)	-1960	-1997

Table 7: Steady state results from ADAMS and MATLAB/Simulink for the nonlinear single-track model.

Transient

TRANSIENT	
	Simulink
$\dot{\psi}$ (deg/s)	-4.77
α_{fT} (deg)	-12.7
α_{rT} (deg)	-1.18
F_{yT} (N)	-2345
F_{yT} (N)	-1997

Table 8: Steady-state results from ADAMS and MATLAB/Simulink for the tyre transient single-track model.

Double-track

DOUBLE-TRACK	
	Simulink
$\dot{\psi} \text{ (deg/s)}$	-4.53
$\alpha_{fr} \text{ (deg)}$	-12.6
$\alpha_{fl} \text{ (deg)}$	-12.6
$\alpha_{rr} \text{ (deg)}$	-1.02
$\alpha_{rl} \text{ (deg)}$	-1.01
$F_{y,fr} \text{ (N)}$	-1010
$F_{y,fl} \text{ (N)}$	-1218
$F_{y,rr} \text{ (N)}$	-869.6
$F_{y,rl} \text{ (N)}$	-1028

Table 9: Steady-state results from ADAMS and MATLAB/Simulink for the double-track model.

Appendix B

The development of bicycle model was carried out taking the Nissan GTR-R35 Nismo as reference.



Figure 32: Nissan GTR-R35 Nismo.

Relevant vehicle parameters, most of them extracted from Nissan Motor Corporation, (2017), are reported in Table 10.

Parameter	Value	Unit
Wheelbase	2780	mm
Height (mm)	1370	mm
Front Track Width (mm)	1600	mm
Rear Track Width (mm)	1600	mm
Curb Weight (kg)	1739	kg
Weight Distribution	54/46	-
Front Tyres Cornering Stiffness	1099.1	N/deg
Rear Tyres Cornering Stiffness	1716.9	N/deg
Yaw Inertia	3508	Kgm ²
Drag Coefficient	0.26	-
Frontal Area	2.1	m ²

Table 10: Nissan GTR-R35 Nismo parameters.

Since tyre cornering stiffness values are not directly provided by the supplier, an algorithm was implemented in MATLAB to estimate them (see Appendix H for the MATLAB code). The method is based on the shape of cornering stiffness-vertical load curve evaluated in Coursework 1. The curve was firstly normalized between 0 and 1 on the y-axis, preserving its shape while allowing its application to other vehicle configurations. Then, to rescale and adapt the curve to the considered vehicle, at least one reference data point is required. Mitsunori, (2021) provided two values of cornering stiffness for a Nissan vehicle with parameters similar to those of the Nissan GT-R, which are reported in Table 11, along with vehicle's dimensional parameters.

Parameter	Value	Unit
Mass	1670	kg

Yaw Inertia	2600	kgm ²
Front Axle – CG	1080	mm
Rear Axle – CG	1620	mm
Front Cornering Stiffness	62.5	kN/rad
Rear Cornering Stiffness	95.5	kN/rad

Table 11: Reference vehicle parameter.

From centre of gravity position information, the vertical load on each tyre was evaluated. Using these load values along with the cornering stiffness data points for both front and rear tyre, the cornering stiffness-vertical load curve was rescaled imposing the passage from the known points. Figure 33 and Figure 34 shows the relationships between cornering stiffness and vertical load of front and rear tyres obtained through the described procedure.

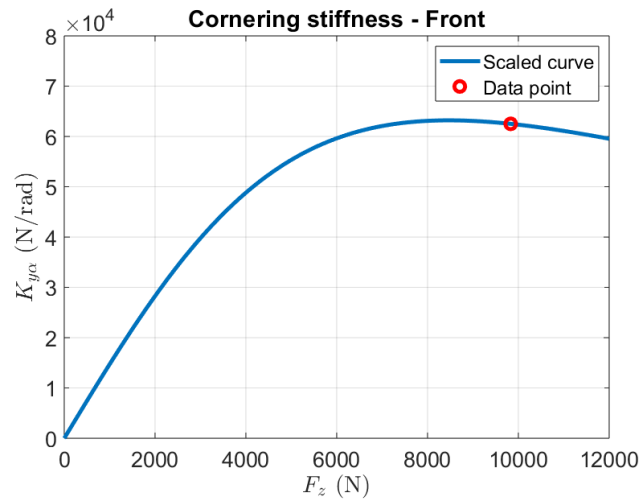


Figure 33: Front tyre cornering stiffness-vertical load relationship.

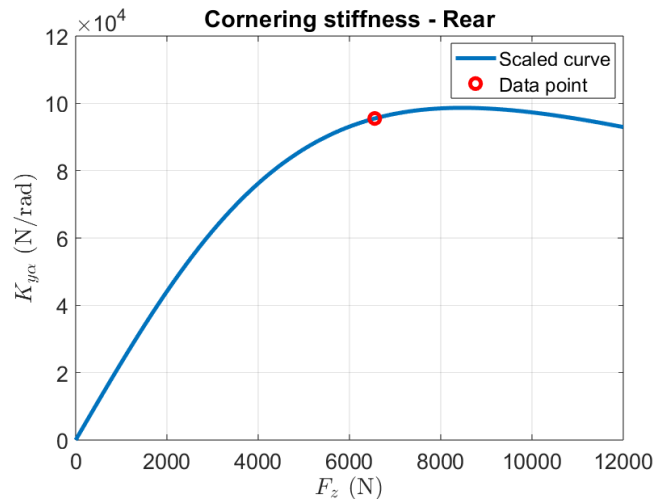


Figure 34: Rear tyre cornering stiffness-vertical load relationship.

Regarding yaw inertia evaluation, since reliable data are not available, the Bixel equation has been exploited (MacInnis *et al.*, 1997):

$$I_{zz} = \frac{mtl}{K} \quad (17)$$

where m is the vehicle mass, t the track width, l the wheelbase and K is the Bixel's approximation constant, which for the studied vehicle corresponds to $K = 2.2048$, as reported in Figure 35.

Vehicle Class	Pitch	Roll	Yaw
Passenger Cars	5.2901	7.9846	2.1942
Sport Utility, $l < 2.41$ m	4.2193	9.4212	2.2048
Sport Utility, $l > 2.41$ m	3.4510	9.4212	2.2048
Pickups	3.3783	9.4738	2.1858
Vans	3.4734	7.8854	2.2168

Figure 35: Bixel's approximation constants for different vehicle types (MacInnis *et al.*, 1997).

As explained in Section 2.1.1, changing the centre of gravity of vehicle impacts on the tyres' cornering stiffness, as it changes the vertical load on the axle. The values of cornering stiffness for the understeer, oversteer and neutral-steer vehicle evaluated from the curve obtained using the methodology explained in this appendix are reported in Table 12.

	US	OS	NS
C_f (N/deg)	1099.1	976.6	1068.3
C_r (N/deg)	1716.9	1624.8	1683

Table 12: Cornering stiffness for understeer, oversteer and neutral-steer vehicle.

Appendix C

The vertical dynamics analysis using quarter-car and half-car models has been carried out taking as reference the Ford Granada.



Figure 36: Ford Granada.

The important parameters needed for models' development, extracted from (Crolla, 1991), are reported in the following tables.

1-DOF Quarter-Car Model		
Parameter	Value	Unit
Sprung mass	317.5	Kg
Spring stiffness	19.74	kN/m
Damping coefficient	1.5	kNs/m

Table 13: Ford Granada 1-DOF Quarter-car model parameters.

2 DOFs Quarter-Car Model		
Parameter	Value	Unit
Sprung mass	317.5	Kg
Unsprung mass	45.4	Kg
Spring stiffness	22	kN/m
Tyre stiffness	192	kN/m
Damping coefficient	1.5	kNs/m

Table 14: Ford Granada 2-DOFs Quarter-car model parameters.

4-DOFs Half-Car Model		
Parameter	Value	Unit
Wheelbase	2760	mm
Front Axle – CG	1250	mm
Rear Axle – CG	1510	mm
Half sprung mass	690	kg

Front unsprung mass	40.5	Kg
Rear unsprung mass	45.4	Kg
Pitch Inertia	1222	Kgm ²
Front spring stiffness	17	kN/m
Rear spring stiffness	22	kN/m
Front tyre stiffness	192	kN/m
Rear tyre stiffness	192	kN/m
Front damping coefficient	1.5	kNs/m
Front damping coefficient	1.5	kNs/m

Table 15: Ford Granada 4-DOFs Half-car model parameters.

Since the 1-DOF quarter-car parameter are not directly provided, they were derived from the 2-DOFs quarter-car model. The same unsprung mass and damping coefficient were used, while the suspension stiffness was evaluated using Equation 18, which evaluates the equivalent stiffness of two springs in series, here represented by tyre and suspension stiffnesses.

$$K_{eq} = \left(\frac{1}{K_1} + \frac{1}{K_2} \right)^{-1} \quad (18)$$

Appendix D

This section provides details about the ADAMS and MATLAB/Simulink implementation of the developed bicycle models.

ADAMS

The ADAMS models are implemented through boxes, joints and forces, as shown in Figure 37. The green box represents the front tyre, the red box the rear tyre and the light-blue box the vehicle body. Four red arrows indicate the lateral forces generated by tyres and the driving and drag forces applied to the vehicle body. Since the simulation runs at constant speed, driving and drag forces are equal and evaluated using Equation 19.

$$F_d = \frac{1}{2} \rho A C_d V^2 \quad (19)$$

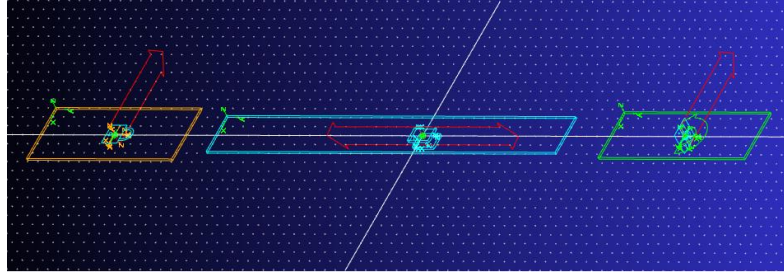


Figure 37: Bicycle model on ADAMS.

MATLAB/Simulink

All the implemented models share the same overall Simulink architecture, shown in Figure 38. The main *vehicle lateral dynamics* block computes the vehicle's lateral behaviour, taking constant longitudinal velocity and a step steering as inputs, and producing relevant output variables to assess lateral behaviour. The steering input is implemented as a step steer with a short transient to reproduce a more realistic manoeuvre. Figure 39 shows the constitutive sub-blocks: the *State equations* block (yellow), the *Side-slip-angles* block (green) and *Lateral-forces* block (light blue). The *State equations* block, identical for all vehicle models, implements the differential equations describing vehicle's longitudinal and lateral dynamics reported in Equation 1.

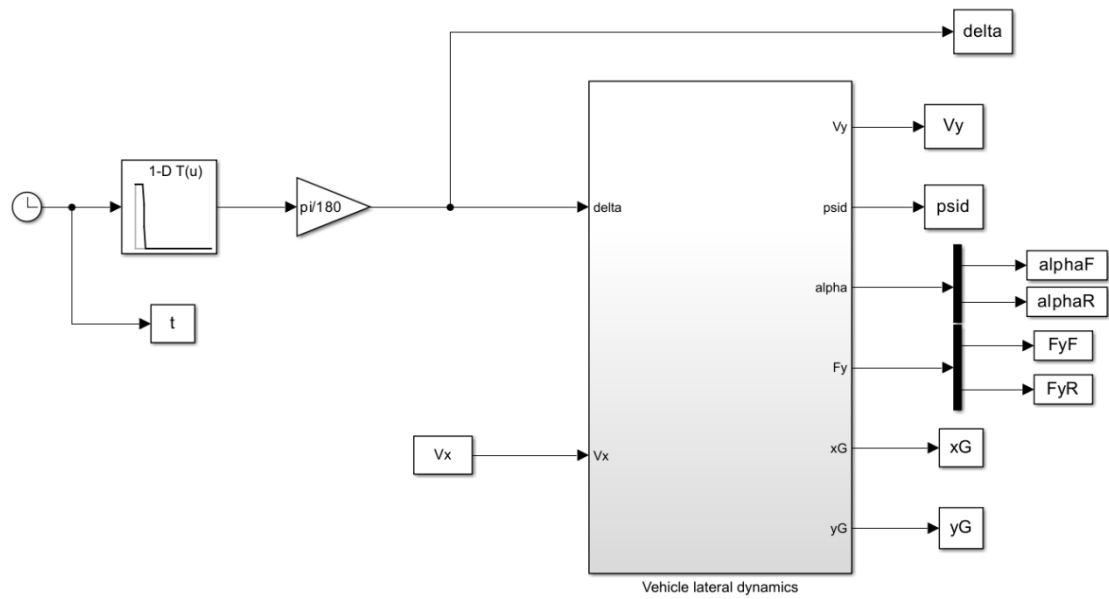


Figure 38: Base Simulink model used to evaluate lateral behaviour of the vehicle under different operating conditions.

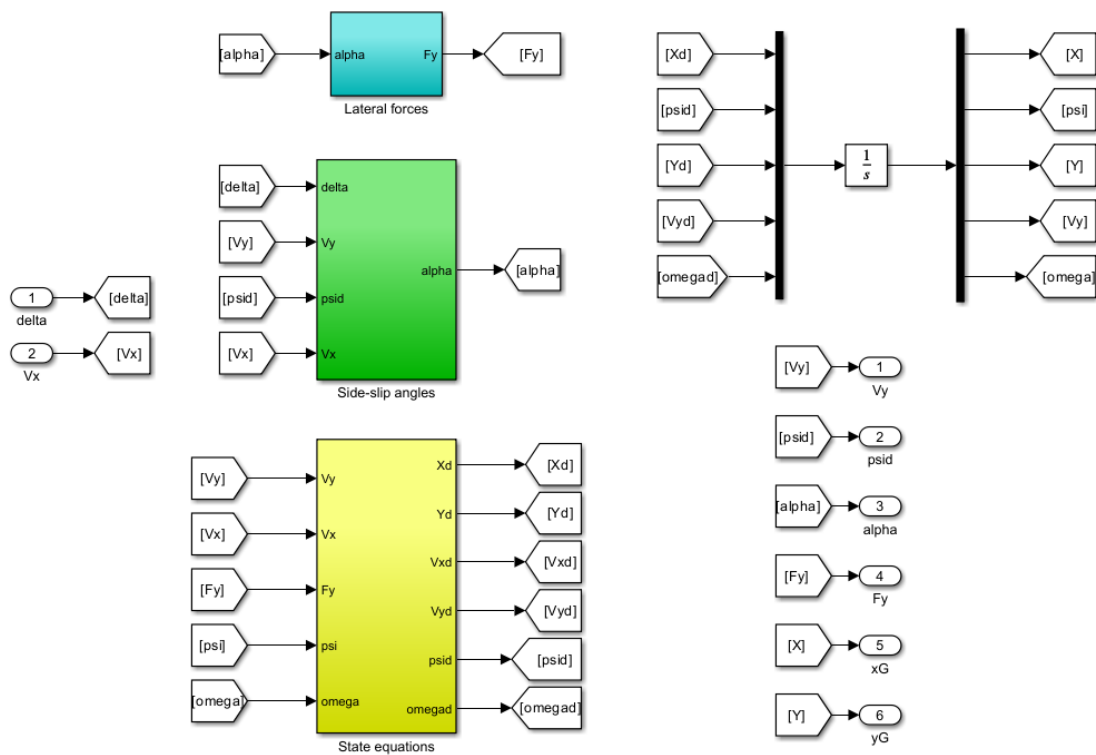


Figure 39: Vehicle lateral dynamics block.

Figure 40 shows the schematic block diagram used to implement state equations.

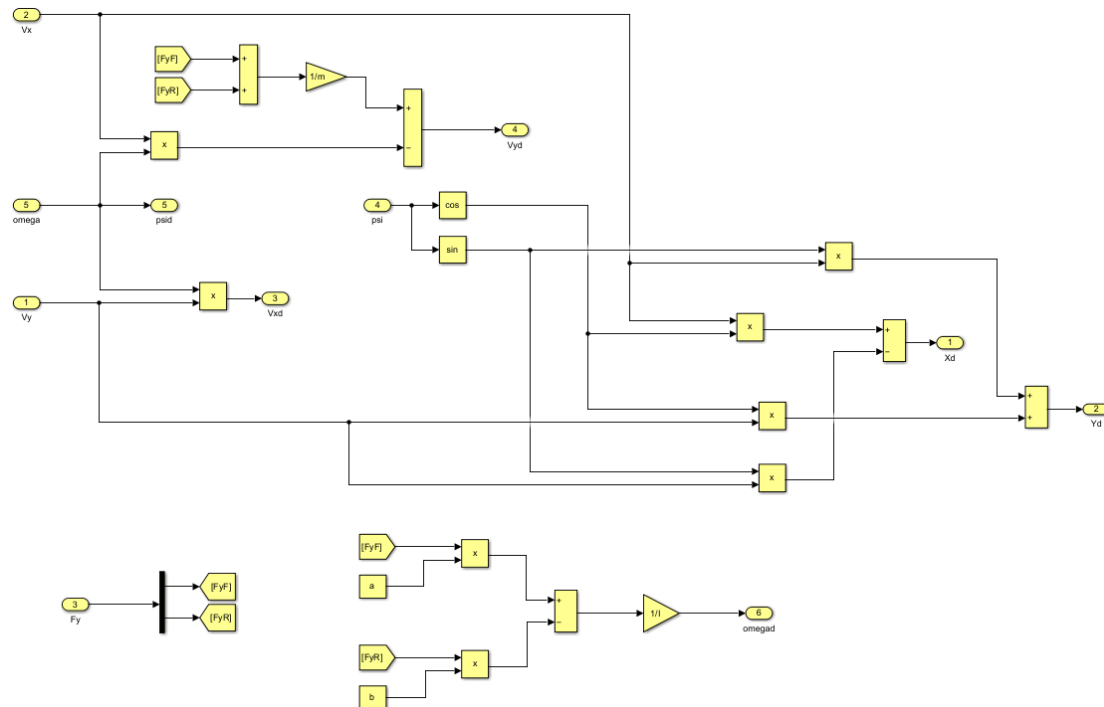


Figure 40: State equations block.

The *Side-slip angles* block evaluates tyre side-slip angles using the linearized relations reported in Equation 3. It is common to all models, except the one implementing lateral load transfer.

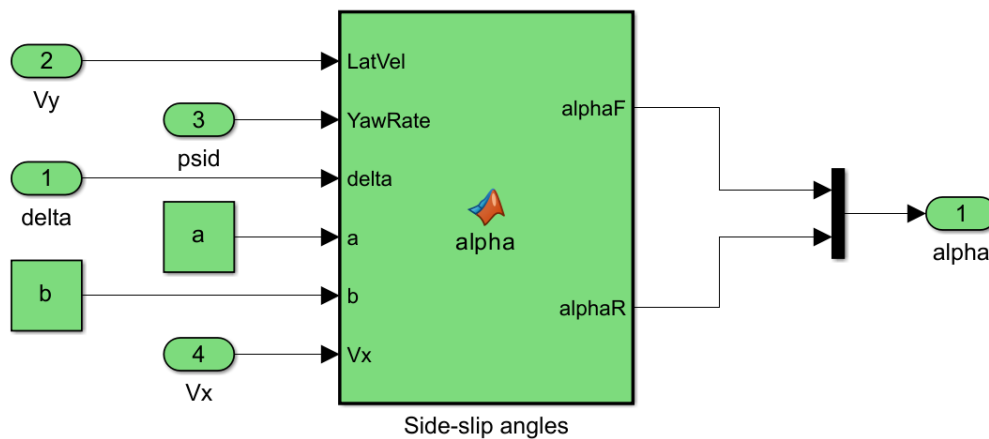


Figure 41: Side-slip angles evaluation block.

The main differences between linear, nonlinear, transient and double-track models lie in the lateral force block, with implementation details provided in the dedicated sections for each simulation.

Linear

The linear vehicle model evaluates the linear lateral forces using a linear relationship with the side slip angle, as described by Equation 2. This is implemented through the dedicated block shown in Figure 42.

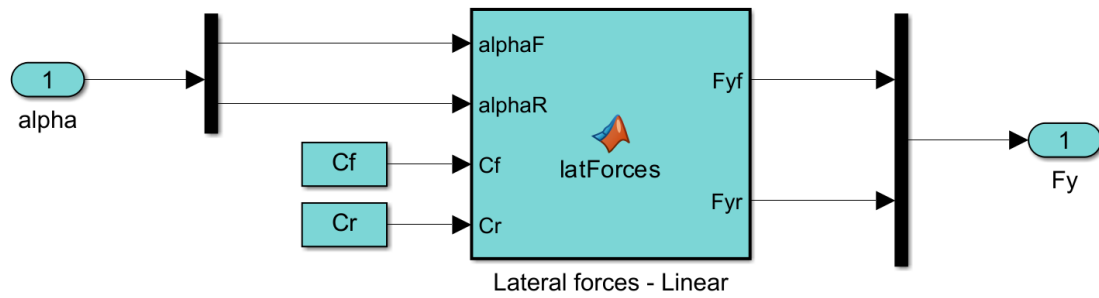


Figure 42: Linear lateral forces evaluation block.

The different load distribution configurations, hence understeer, oversteer and neutral steer, are modelled changing a *loadDist* variable. More details about the code are available in Appendix...

Nonlinear

In the MATLAB/Simulink nonlinear model, lateral forces are evaluated using Look-Up Tables that implement the nonlinear relationship between lateral-force and side-slip angle. As shown in Figure 43, the dedicated block outputs the lateral forces evaluated through the LUTs, which take side slip angles as inputs.

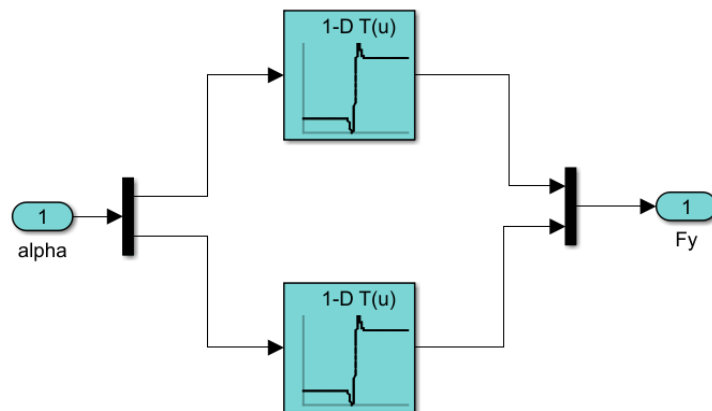


Figure 43: Nonlinear lateral forces evaluation block.

Transient

The *Tyre transient dynamics* block computes lateral forces using Equation 4 to capture transient effects. Figure 44 depicts the Simulink implementation.

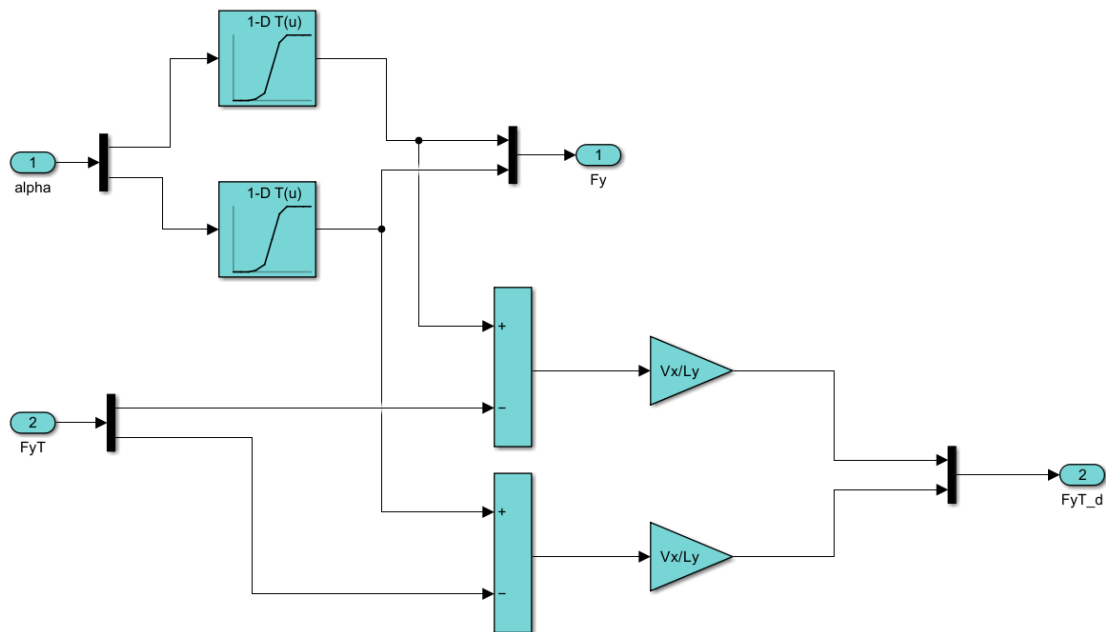


Figure 44: Transient lateral forces evaluation block.

Double-Track

The last model considered is the one that accounts for axle load transfer during cornering. An additional block is introduced, as shown in Figure 45, which updates the cornering stiffness value according to the vertical load acting on each tyre as the simulation proceeds.

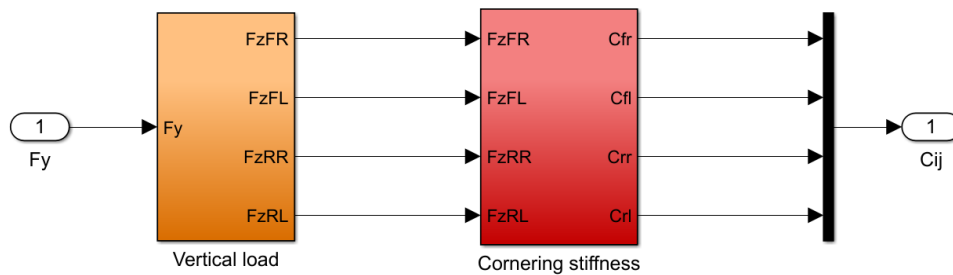


Figure 45: Load transfer evaluation blocks.

The *Vertical load* block shown in Figure 46 evaluates the load on each tyre, considering the static load and the load transfer contribution evaluated through Equation 5.

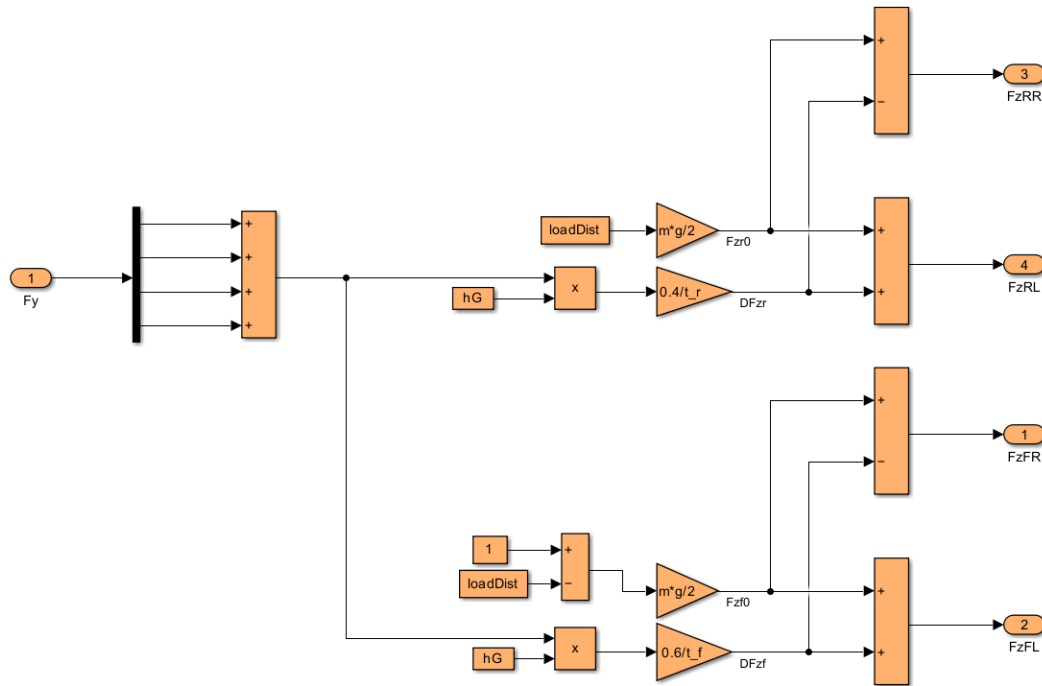


Figure 46: Vertical load block.

The *Cornering stiffness* block, reported in Figure 47, updates at each iteration the value of the cornering stiffness according to the curves shown in Figure 33 and 34.

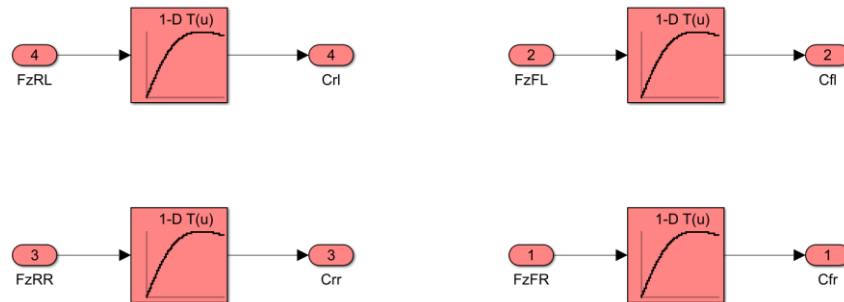


Figure 47: Cornering stiffness block.

The *Side-slip angles* block is modified to include the effect of track width and the different steering angles of front tyres, as stated by Equation 6 and Equation 7.

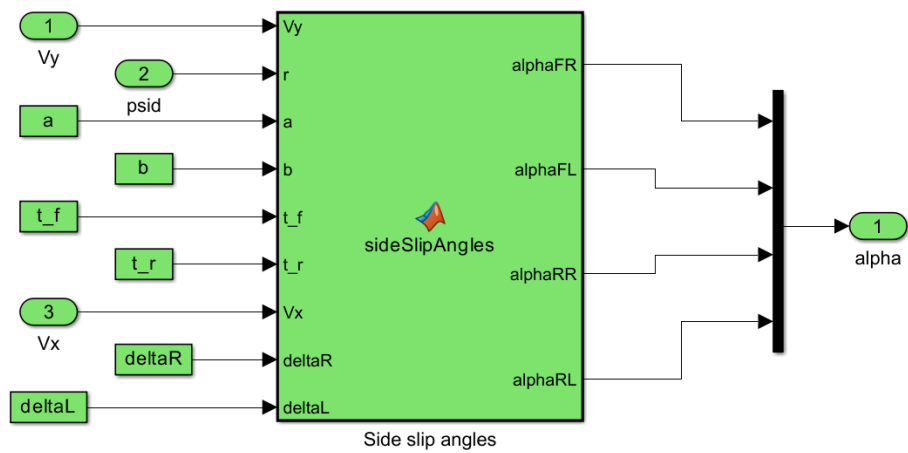


Figure 48: Side-slip angles evaluation block - Double-track.

Finally, the *Lateral forces* block computes the force on each tyre by scaling the normalized spline output with the cornering stiffness provided by the *Cornering Stiffness* block.

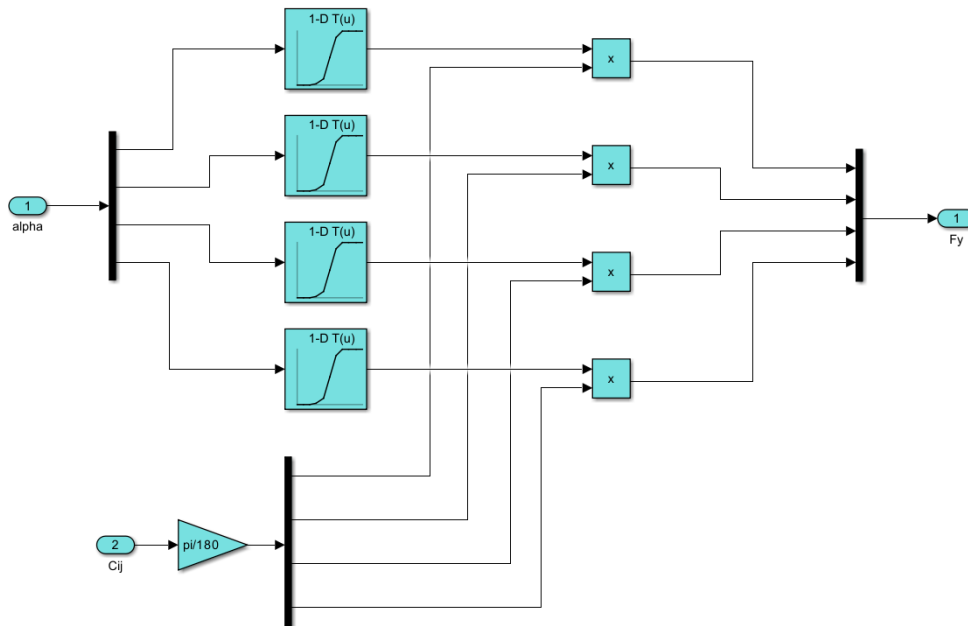


Figure 49: Lateral forces block - Double track.

Appendix E

This section provides insights into the ADAMS and MATLAB/Simulink implementation of models addressing vehicle vertical motion

ADAMS

The ADAMS vertical-motion models were built using boxes with arbitrary dimensions but real mass and inertia values, reported in Appendix C, connected by translational and rotational joints and a built-in spring-damper element, which represents the suspension. This setup allows spring-damper connections between boxes to model road-to-unsprung and unsprung-to-sprung links, using specified stiffness and damping values. Both linear and nonlinear dampers can be implemented, with nonlinear behaviour defined via a spline.

Since ADAMS does not allow the implementation of a single joint with only pitch and heave degrees of freedom, the half-car model was realized by introducing a dummy body. This solution enables the use of two separate joints to represent these motions, as shown in Figure 50.

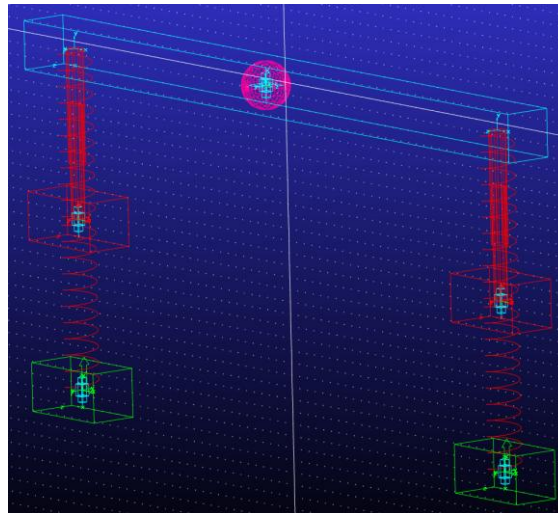


Figure 50: Half-car model in ADAMS.

MATLAB/Simulink

The overall block architecture is the same for all three models, each featuring an input signal and several outputs, depending on the specific model, to assess the vertical performance of the suspension system. The main difference between the models lies in the block that implements the characteristic differential equations of each model.

1-DOF Quarter-Car Model

The overall block diagram of the simplest 1-DOF quarter-car model is depicted in Figure 51. The represented input is a step signal, used to simulate a bump. Figure 52 shows the internal block diagram of the *1DOF Quarter-Car Model*

block, highlighting the block implementation of the state equation reported in Equation 10.

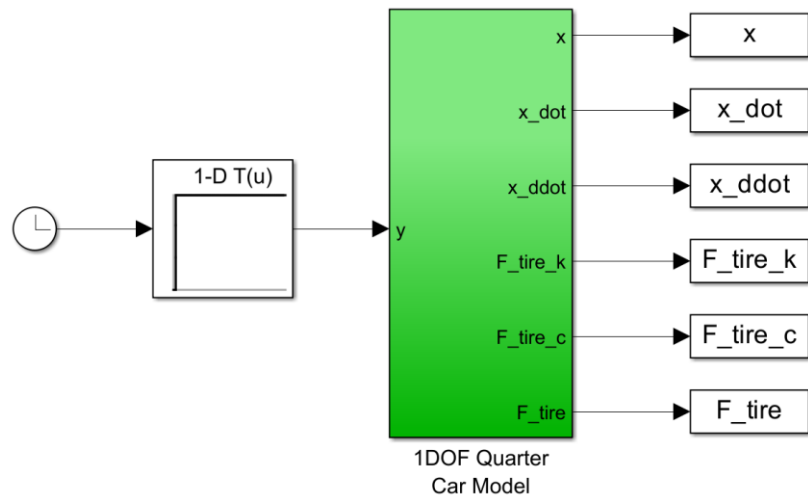


Figure 51: 1-DOF quarter-car model overall Simulink architecture.

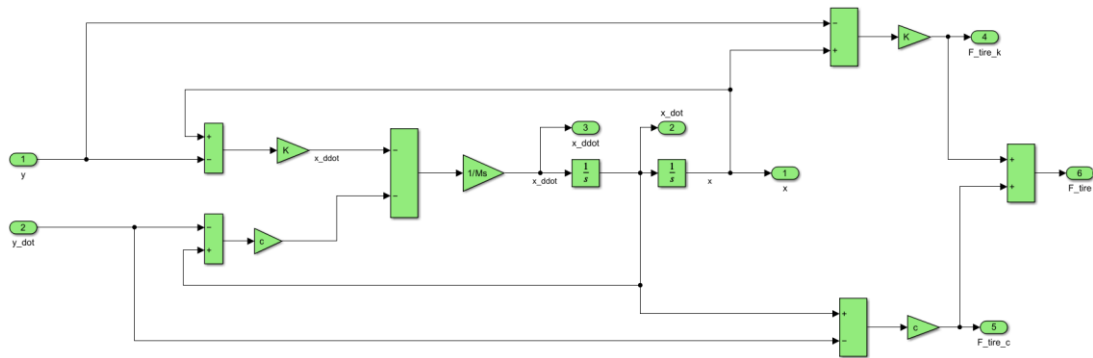


Figure 52: 1-DOF quarter-car model Simulink state equations block diagram.

2-DOFs Quarter-Car Model

The differences in the 2-DOFs quarter-car model lie in the block outputs and the internal structure of the overall block, which instead implements the differential equations reported in Equation 13, as shown by Figure 54.

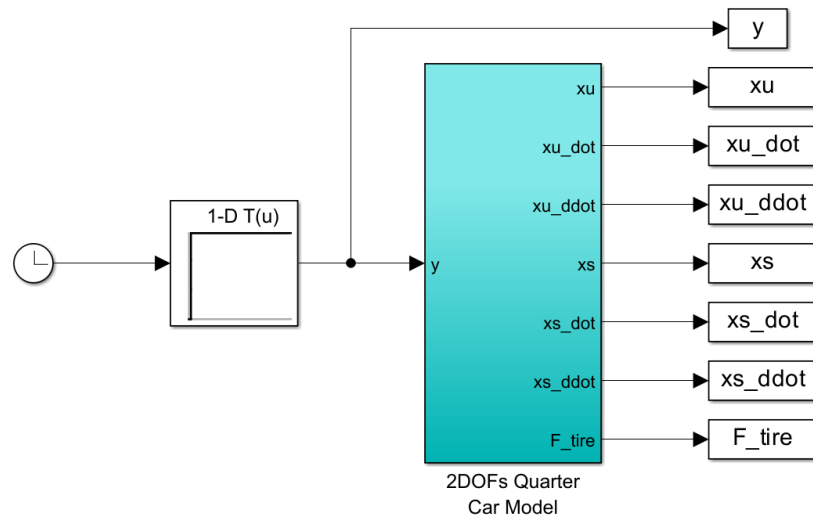


Figure 53: 2-DOFs quarter-car model overall Simulink architecture.

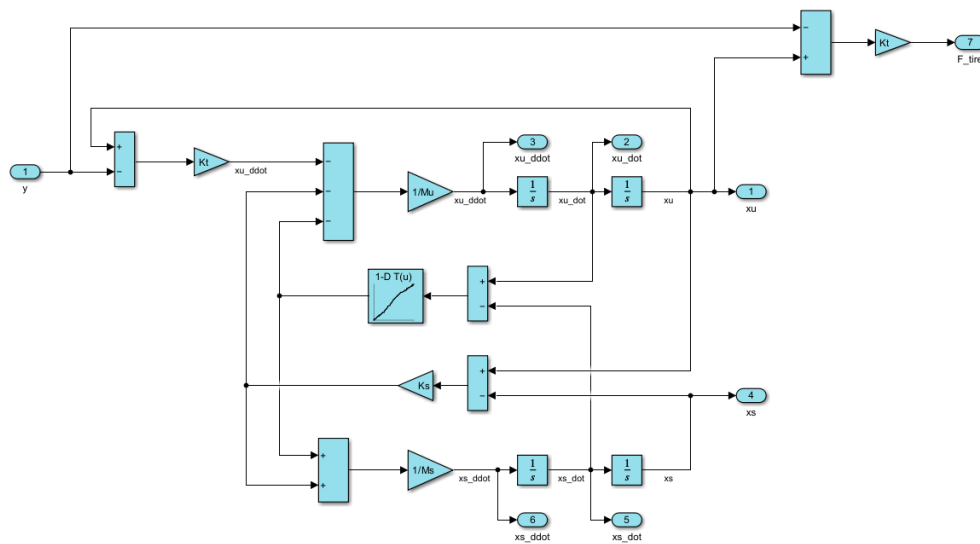


Figure 54: 2-DOFs quarter-car model Simulink state equations block diagram.

Half-Car Model

The half-car model, reported in Figure 55, shares the same overall architecture as the two quarter-car models, with the key difference that the overall block has two inputs. Both inputs are generated from the same signal, however the rear input is delayed by an amount proportional to the wheelbase divided by the vehicle speed. The main block implements the differential equations reported in Equation 14, as shown in Figure 56.

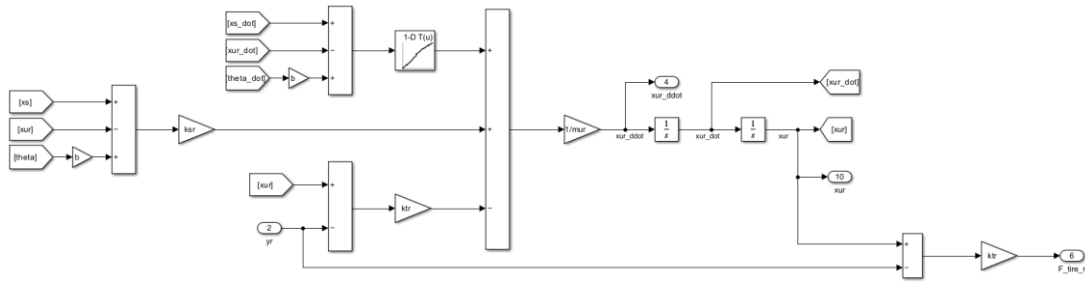


Figure 56: 4-DOFs half-car model Simulink state equations block diagram.

To account for ISO standards related to riding comfort and road input, two additional blocks have been implemented, shown in blue and red in Figure 55. These blocks represent the ISO 8606 and ISO 2631 standards, modeled through transfer functions in the Laplace domain. ISO 2631 is an international standard for evaluating the effects of vibrations on humans. It indicates that the human body is particularly sensitive to vibrations in the 4-8 Hz range. Consequently, the implemented transfer function amplifies vibrations within this frequency range while filtering out others, allowing for a more realistic assessment of passenger-perceived comfort. The ISO 2631 transfer function is given by (Tonoli, 2024):

$$H_{2631}(s) = \frac{80.03s^2 + 989s + 0.02108}{s^3 + 78.92s^2 + 2412s + 5614} \quad (20)$$

The blue transfer function, used to evaluate road irregularities, is expressed as (Amati, 2025):

$$H(s) = \frac{2\pi\sqrt{G_rV}}{s + s_0} \quad (21)$$

Here, V is the longitudinal vehicle speed, s_0 is a pole added to prevent infinite gain at low frequencies and G_r is a parameter defining road roughness according to ISO 8606. For this study, class B roads were selected, as most European roads fall into this category. The corresponding mean G_r value, obtained from Table 16, is $G_r = 6.4 \times 10^{-7}$.

To generate the road profile, the following property of the Power Spectral Density (PSD) is exploited:

$$PSD_{out} = |H| \cdot PSD_{in} \quad (22)$$

To obtain a road profile whose PSD matches the ISO 8606 standard, an input signal with $PSD_{in} = 1$ should be used. For this reason, a white noise signal is provided as input to the transfer function, producing an output PSD consistent with real road conditions (Amati, 2025).

	G_{rmin}	G_{rmean}	G_{rmax}
Class	$(\text{m}^2\text{cycles/m})$	$(\text{m}^2\text{cycles/m})$	$(\text{m}^2\text{cycles/m})$
A	—	1.6×10^{-7}	3.2×10^{-7}
B	3.2×10^{-7}	6.4×10^{-7}	1.28×10^{-6}
C	1.28×10^{-6}	2.56×10^{-6}	5.12×10^{-6}
D	5.12×10^{-6}	1.024×10^{-5}	2.048×10^{-5}
E	2.048×10^{-5}	4.096×10^{-5}	8.192×10^{-5}
F	8.192×10^{-5}	1.6384×10^{-4}	3.2768×10^{-4}
G	3.2768×10^{-4}	6.5536×10^{-4}	1.31072×10^{-3}
H	1.31072×10^{-3}	2.62144×10^{-3}	—

Table 16: Minimum, average and maximum values of G_r for different classes of road (Amati, 2025).

Appendix F

This section provides the detailed procedure used to determine the equations governing the natural frequencies of the 2-DOFs quarter-car model. These equations were employed to select the range of stiffness values during the optimisation process, to ensure that the first resonance frequency of the system remained between 1.15 and 1.35 Hz. For more details about MATLAB implementation refer to Appendix H.

Starting from the equation of motions of the 2-DOFs model, to assess the natural frequencies, zero damping and a harmonic input are considered:

$$\begin{aligned}
 & \begin{cases} m_s \ddot{x}_s + k_s(x_s - x_u) = 0 \\ m_u \ddot{x}_u + k_s(x_u - x_s) + k_t(x_u - y) = 0 \end{cases} \rightarrow \begin{cases} (k_s - \omega^2 m_s)x_s - k_s x_u = 0 \\ -k_s x_s + (k_s + k_t - \omega^2 m_u)x_u = 0 \end{cases} \\
 & \rightarrow \begin{cases} \frac{x_s}{x_u} = \frac{k_s}{k_s - \omega^2 m_s} \\ \frac{x_s}{x_u} = \frac{k_s + k_t - \omega^2 m_u}{k_s} \end{cases} \rightarrow \frac{k_s}{k_s - \omega^2 m_s} = \frac{k_s + k_t - \omega^2 m_u}{k_s} \\
 & \rightarrow k_s^2 = (k_s + k_t - \omega^2 m_u)(k_s - \omega^2 m_s) \\
 & \rightarrow k_s^2 + k_s k_t - m_u k_s \omega^2 - m_s k_s \omega^2 - m_s k_t \omega^2 + m_s m_u \omega^4 - k_s^2 = 0 \\
 & \rightarrow m_s m_u \omega^4 - (m_u k_s + m_s k_s + m_s k_t) \omega^2 + k_s k_t = 0 \\
 & \rightarrow \omega_{1,2} = \sqrt{\frac{(m_u k_s + m_s k_s + m_s k_t) \pm \sqrt{(m_u k_s + m_s k_s + m_s k_t)^2 - 4 m_s m_u k_s k_t}}{2 m_s m_u}}
 \end{aligned}$$

Appendix G

Derivative Analysis

Vehicle cornering response can also be assessed analytically using derivatives of stability, which represent the forces and moments derivatives with respect to the variables β , r and δ , as shown in Equation 23. In a simplified lateral analysis, neglecting aerodynamic forces and aligning torques, Y_β , Y_δ , N_β , and N_δ are constant, while Y_r and N_r vary with $1/V$ (Genta and Morello, 2020).

$$\begin{cases} mV(r + \dot{\beta}) = Y_\beta\beta + Y_rr + Y_\delta\delta \\ I\dot{r} = N_\beta\beta + N_rr + N_\delta\delta \end{cases} \quad (23)$$

Figure 57 shows how analytical solution of motion equations yields consistent cornering responses if compared to the steady-state yaw-rate value evaluated through numerical approaches and the analytical one evaluated as follows:

$$r_{ss} = C_2 \cdot \delta / k$$

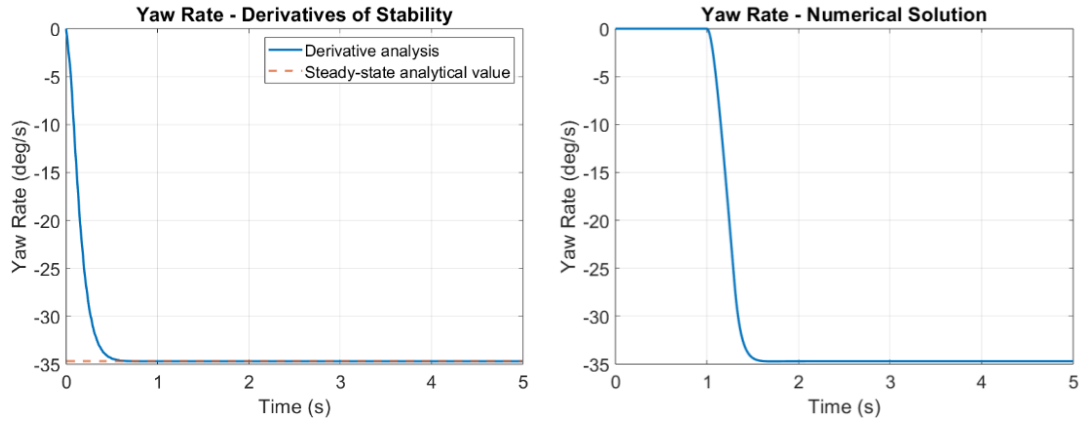


Figure 57: Derivative analysis results.

Appendix H

Code to evaluate cornering stiffness curves.

```
clear
close all
clc

load('Kya_Fz.mat');
Fz_fit = Fz_fit';
Kya_fit = Kya_fit';

loadDist = 0.46;           % load distribution rear (-)
m = 1739;                  % vehicle mass (kg)
g = 9.81;                  % gravity acceleration (m/s^2)

%% Front
KyaF_data = 62500;
FzF_data = 9830;
FzF_GTR = (1 - loadDist) * m * g;
KyaF_norm = normalize(Kya_fit, 'range');
KyaF_data_norm = interp1(Fz_fit, KyaF_norm, FzF_data, 'linear');
KyaF_scaled = KyaF_norm * KyaF_data/KyaF_data_norm;
Cf = interp1(Fz_fit, KyaF_scaled, FzF_GTR, 'linear');           % [N/rad]
Cf = Cf * pi/180;                                               % [N/deg]

% Plot
figure('Position', [100 100 700 500]);
plot(Fz_fit, KyaF_scaled, LineWidth=3);
hold on;
plot(FzF_data, KyaF_data, 'bo', color='r', MarkerSize=8, LineWidth=3);
xlabel('$F_z$ (N)', Interpreter='latex');
ylabel('$K_{y \alpha}$ (N/rad)', Interpreter='latex');
ylim([0, 80000]);
title('Cornering stiffness - Front');
legend('Scaled curve', 'Data point');
grid on;

%% Rear
KyaR_data = 95500;
FzR_data = 6553;
FzR_GTR = loadDist * m * g;
KyaR_norm = normalize(Kya_fit, 'range');
KyaR_data_norm = interp1(Fz_fit, KyaR_norm, FzR_data, 'linear');
KyaR_scaled = KyaR_norm * KyaR_data/KyaR_data_norm;
Cr = interp1(Fz_fit, KyaR_scaled, FzR_GTR, 'linear');           % [N/rad]
Cr = Cr * pi/180;                                               % [N/deg]

% Plot
figure('Position', [100 100 700 500]);
plot(Fz_fit, KyaR_scaled, LineWidth=3);
hold on;
plot(FzR_data, KyaR_data, 'bo', color='r', MarkerSize=8, LineWidth=3);
xlabel('$F_z$ (N)', Interpreter='latex');
ylabel('$K_{y \alpha}$ (N/rad)', Interpreter='latex');
ylim([0, 120000]);
title('Cornering stiffness - Rear');
legend('Scaled curve', 'Data point');
grid on;
```

```

save('CornStiff_NissanGTR', 'Cf', 'Cr');
save('Load_distribution', 'loadDist');
save('Kya_Fz_fit', 'Fz_fit', 'KyaF_scaled', 'KyaR_scaled');

```

Code to assess lateral behaviour with linear tyre model.

```

clear
close all
clc

set(0, 'DefaultAxesFontSize', 16);
set(0, 'DefaultTextFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

% Vehicle parameters
load('Load_distribution.mat');
load('CornStiff_NissanGTR.mat');
Cf = Cf*180/pi;           % front cornering stiffness (N/rad)
Cr = Cr*180/pi;           % rear cornering stiffness (N/rad)
l = 2.78;                 % wheelbase (m)
a = loadDist*l;           % CG - front axle (m)
b = (1-loadDist)*l;       % CG - rear axle (m)
m = 1739;                 % vehicle mass (kg)
Vx = 10;                  % vehicle speed (m/s)
I = 3508;                 % yaw inertia (kgm^2)

% Initial conditions
zita0 = [0, 0, 0, 0, 0];

% Simulation parameters
Tsim = 10;
% Step Steer
t = 0:0.1:Tsim;
delta = interp1([ 0, 1, 1.3, Tsim],[0, 0, -12, -12],t);
dt = 1e-4;
dt_save = 1e-2;

% Simulation
file = 'Lateral_Linear_sim.slx';
% open(file);
sim(file);

% Kus
Kus = m/l * (b/Cf - a/Cr);

% Curvature Gain
V = linspace(0, 70, 100);
curvGain = 1./(1 + Kus * V.^2);
% save('curveGain_OS', 'V', 'curvGain');

%% Plots
% Curvature gain
figure('Position', [100 100 700 500]);
plot(V, curvGain, LineWidth=2);
xlabel('Vehicle Speed (m/s)');
ylabel('$\frac{\rho_{ss}}{\delta_f}$', 'Interpreter','latex', FontSize=28);
title('Curvature Gain - Linear')

```

```

% xlim([0 50]);
% ylim([-40 40]);
grid on;

% delta vs V^2/R (steady state acceleration)
R = 25;
delta = 1./R + Kus * V.^2/R;
figure('Position', [100 100 700 500]);
plot(V.^2/R, delta, LineWidth=2);
xlabel('$V^2/R$', 'Interpreter','latex', FontSize=16);
ylabel('$\delta_f$', 'Interpreter','latex', FontSize=16);
grid on;

% Lateral Velocity - Yaw Rate
figure('Position', [100 100 700 500]);
% yyaxis left
% plot(t, Vy, 'LineWidth', 2);
% ylabel('Lateral Velocity (m/s)');
% yyaxis right
plot(t, psid*180/pi, 'LineWidth', 2, Color=[0.8500, 0.3250, 0.0980]);
ylabel('Yaw Rate (deg/s)');
xlabel('Time (s)');
% ylim([-45 0]);
% legend('Lateral Velocity', 'Yaw Rate');
title('Yaw Rate - Linear');
grid on;

% Side-slip angles
figure('Position', [100 100 700 500]);
plot(t, alphaF, LineWidth=2);
hold on;
plot(t, alphaR, LineWidth=2);
xlabel('Time (s)');
ylabel('Side-Slip Angle (deg)');
legend('\alpha_f', '\alpha_r');
title('Front-Rear Side-slip angles - Linear');
grid on;

% Lateral forces
figure('Position', [100 100 700 500]);
plot(t, FyF, LineWidth=2);
hold on;
plot(t, FyR, LineWidth=2);
xlabel('Time (s)');
ylabel('Lateral Force (N)');
legend('F_{yf}', 'F_{yr}');
title('Front-Rear Lateral forces - Linear');
grid on;
ax = gca;
ax.YAxis.Exponent = 0;

% Trajectory
figure('Position', [100 100 700 500]);
plot(xG, yG, LineWidth=2);
xlabel('X_G (m)');
ylabel('Y_G (m)');
title('Vehicle Trajectory - Linear');
grid on;
% save('Trajectory_NS', 'xG', 'yG');

```

```

save('Trajectory_Linear', 'xG', 'yG');
save('Linear', 't', 'psid', 'FyF', 'FyR', 'alphaF', 'alphaR');

```

Code to evaluate trajectory differences between understeer, oversteer and neutral-steer.

```

clear
close all
clc

US = load("Trajectory_US.mat");
OS = load("Trajectory_OS.mat");
NS = load("Trajectory_NS.mat");

figure('Position', [100 100 700 500]);
plot(-US.yG, US.xG, LineWidth=2);
hold on;
plot(-OS.yG, OS.xG, LineWidth=2);
hold on;
plot(-NS.yG, NS.xG, LineWidth=2);
xlabel('x_G (m)');
ylabel('y_G (m)');
title('Trajectory - US, OS, NS');
axis equal;
legend('US', 'OS', 'NS');
grid on;

```

Code to evaluated curvature gain differences between understeer, oversteer and neutral-steer.

```

clear
close all
clc

US = load("curveGain_US.mat");
OS = load("curveGain_OS.mat");
NS = load("curveGain_NS.mat");

figure('Position', [100 100 700 500]);
plot(US.V, US.curvGain, LineWidth=2, Color=[0.8500, 0.3250, 0.0980]);
hold on;
plot(NS.V, NS.curvGain, LineWidth=2, Color=[0.9290, 0.6940, 0.1250]);
hold on;
plot(OS.V(1:39), OS.curvGain(1:39), LineWidth=2, Color=[0, 0.4470, 0.7410]);
hold on;
plot(OS.V(40:end), OS.curvGain(40:end), LineWidth=2, Color=[0, 0.4470, 0.7410]);
hold on;
xline(27.2198, '--k', LineWidth=2, Color=[0, 0.4470, 0.7410]);
xlabel('Vehicle Speed (m/s)');
ylabel('$\frac{\rho_{ss}}{\delta_f}$', 'Interpreter','latex', FontSize=28);
title('Curvature Gain Comparison - US, OS, NS');
xlim([0 50]);
ylim([-5 5]);
legend('US', 'NS', 'OS');
grid on;

```

Code to create the LUT for the lateral dynamics assessment with nonlinear tyre model.

```
clear
close all
clc

% Vehicle parameters
load('..\Lateral_Linear\CornStiff_NissanGTR.mat');
load('..\Lateral_Linear\Load_distribution.mat');
Cf = Cf*180/pi;           % front axle cornering stiffness (N/rad)
Cr = Cr*180/pi;           % rear axle cornering stiffness (N/rad)
l = 2.78;                 % wheelbase (m)
a = loadDist*l;           % CG - front axle (m)
b = (1-loadDist)*l;       % CG - rear axle (m)
m = 1739;                 % vehicle mass (kg)
U = 30;                   % vehicle speed (m/s)
I = 3508;                 % yaw inertia (kgm^2)

% Nonlinear Tire model
splineData = readmatrix("Nonlinear Tyre Model Spline.xlsx");
alpha = splineData(:,1);
Fy_norm = splineData(:,2);
alphaRange = linspace(-90,90,1000);
s = spline(alpha, Fy_norm, alphaRange);
FyR_real = s * Cr * pi/180;
FyF_real = s * Cf * pi/180;

figure('Position', [100 100 700 500]);
plot(alphaRange, FyF_real, LineWidth=3);
xlabel('Side Slip Angle (deg)');
ylabel('Lateral Force (N)');
title('Front Tyre');
grid on;

figure('Position', [100 100 700 500]);
plot(alphaRange, FyR_real, LineWidth=3);
xlabel('Side Slip Angle (deg)');
ylabel('Lateral Force (N)');
title('Rear Tyre');
grid on;

save('spline.mat', "FyF_real", "FyR_real", "alphaRange");
```

Code to assess lateral dynamics analysis using nonlinear tyre model.

```
clear
close all
clc

set(0, 'DefaultAxesFontSize', 16);
set(0, 'DefaultTextFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

g = 9.81;
```

```

% Vehicle parameters
loadDist = 0.46;           % load distribution rear (-)
l = 2.78;                  % wheelbase (m)
a = loadDist*l;           % CG - front axle (m)
b = (1-loadDist)*l;       % CG - rear axle (m)
m = 1739;                 % vehicle mass (kg)
Vx = 30;                  % vehicle speed (m/s)
I = 3508;                 % yaw inertia (kgm^2)

% Load tire nonlinear Fy-alpha curve
load('spline.mat');
FyF_spline = FyF_real';
FyR_spline = FyR_real';
alpha_spline = alphaRange';

% Simulation parameters
Tsim = 100;
% Step Steer
t = 0:0.1:Tsim;
delta = interp1([ 0, 1, 1.3, Tsim],[0, 0, -12, -12],t);
dt = 1e-4;
dt_save = 1e-2;
zita0 = [0, 0, 0, 0, 0];

% Simulation
file = 'Lateral_Nonlinear_sim.slx';
% open(file);
sim(file);

%% Plots
figure('Position', [100 100 700 500]);
plot(xG, yG, LineWidth=2);
xlabel('X_G (m)');
ylabel('Y_G (m)');
axis equal;
title('Vehicle Trajectory - Nonlinear');
grid on;
save('Trajectory_NonLinear', 'xG', 'yG');

figure('Position', [100 100 700 500]);
plot(t, FyF, LineWidth=2);
hold on;
plot(t, FyR, LineWidth=2);
xlabel('Time (s)');
ylabel('Lateral Force (N)');
legend('F_{yf}', 'F_{yr}');
title('Front-Rear Lateral forces - Nonlinear');
grid on;

figure('Position', [100 100 700 500]);
plot(t, alphaF, LineWidth=2);
hold on;
plot(t, alphaR, LineWidth=2);
xlabel('Time (s)');
ylabel('Side-Slip Angle (deg)');
legend('\alpha_f', '\alpha_r');
title('Front-Rear Side-slip angles - Nonlinear');
grid on;

```

```

% Lateral Velocity - Yaw Rate
figure('Position', [100 100 700 500]);
plot(t, psid*180/pi, 'LineWidth', 2, Color=[0.8500, 0.3250, 0.0980]);
ylabel('Yaw Rate (deg/s)');
xlabel('Time (s)');
title('Yaw Rate - Nonlinear');
grid on;

save('Nonlinear', 't', 'psid', 'FyF', 'FyR', 'alphaF', 'alphaR');

```

Code to compare lateral behaviour models with linear and nonlinear tyre model.

```

clear
close all
clc

linear = load('../Lateral_Linear/Linear.mat');
nonlinear = load("Nonlinear.mat");

% Yaw Rate
figure('Position', [100 100 700 500]);
plot(linear.t, linear.psid*180/pi, LineWidth=2);
hold on;
plot(nonlinear.t, nonlinear.psid*180/pi, LineWidth=2);
ylabel('Yaw Rate (deg/s)');
xlabel('Time (s)');
legend('Linear', 'Nonlinear');
title('Yaw Rate - Linear, Nonlinear');
grid on;

% Forces
figure('Position', [100 100 700 500]);
plot(linear.t, linear.FyF, LineWidth=2);
hold on;
plot(linear.t, linear.FyR, LineWidth=2);
hold on;
plot(nonlinear.t, nonlinear.FyF, LineWidth=2);
hold on;
plot(nonlinear.t, nonlinear.FyR, LineWidth=2);
xlabel('Time (s)');
ylabel('Lateral Force (N)');
ylim([-25000 0]);
legend('F_{yf,linear}', 'F_{yr,linear}', 'F_{yf,nonlinear}',
'F_{yr,nonlinear}');
title('Front-Rear Lateral forces - Linear, Nonlinear');
grid on;
ax = gca;
ax.YAxis.Exponent = 0;

% Sideslip angles
figure('Position', [100 100 700 500]);
plot(linear.t, linear.alphaF, LineWidth=2);
hold on;
plot(linear.t, linear.alphaR, LineWidth=2);
hold on;
plot(nonlinear.t, nonlinear.alphaF, LineWidth=2);
hold on;
plot(nonlinear.t, nonlinear.alphaR, LineWidth=2);
xlabel('Time (s)');

```



```

ylabel('Side-Slip Angle (deg)');
legend('\alpha_{f,linear}', '\alpha_{r,linear}', '\alpha_{f,nonlinear}',
'\alpha_{r,nonlinear}');
title('Front-Rear Side-slip angles - Linear, Nonlinear');
grid on;
ax = gca;
ax.YAxis.Exponent = 0;

```

Code to compare trajectory of lateral behaviour models with linear and nonlinear tyre model.

```

clear
close all
clc

lin = load("../Lateral_Linear/Trajectory_Linear.mat");
nonlin = load("Trajectory_Nonlinear.mat");

figure('Position', [100 100 700 500]);
plot(-lin.yG, lin.xG, LineWidth=2);
hold on;
plot(-nonlin.yG, nonlin.xG, LineWidth=2);
xlabel('x_G (m)');
ylabel('y_G (m)');
axis equal;
legend('Linear', 'Nonlinear');
title('Trajectory - Linear, Nonlinear');
grid on;

```

Code to assess lateral behaviour of the vehicle using tyre transient model.

```

clear
close all
clc

set(0, 'DefaultAxesFontSize', 16);
set(0, 'DefaultTextFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

g = 9.81;

% Vehicle parameters
loadDist = 0.46;           % load distribution rear (-)
l = 2.78;                  % wheelbase (m)
a = loadDist*l;            % CG - front axle (m)
b = (1-loadDist)*l;        % CG - rear axle (m)
m = 1739;                  % vehicle mass (kg)
Vx = 30;                   % vehicle speed (m/s)
I = 3508;                  % yaw inertia (kgm^2)
Ly = 0.45;                 % relaxation length (m)

% Load tire nonlinear Fy-alpha curve
load('spline.mat');
FyF_spline = FyF_real';
FyR_spline = FyR_real';
alpha_spline = alphaRange';

% Simulation parameters

```

```

Tsim = 10;
% Step Steer
t = 0:0.1:Tsim;
delta = interp1([ 0, 1, 1.3, Tsim],[0, 0, -12, -12],t);
dt = 1e-4;
dt_save = 1e-2;
zita0 = [0, 0, 0, 0, 0];

% Simulation
file = 'Lateral_Nonlinear_Transient_sim.slx';
% open(file);
sim(file);

%% Plots
figure('Position', [100 100 700 500]);
plot(xG, yG, LineWidth=2);
xlabel('X_G (m)');
ylabel('Y_G (m)');
title('Vehicle Trajectory - Transient');
grid on;

figure('Position', [100 100 700 500]);
plot(t, FyF, LineWidth=2);
hold on;
plot(t, FyR, LineWidth=2);
hold on;
plot(t, FyTF, LineWidth=2);
hold on;
plot(t, FyTR, LineWidth=2);
xlabel('Time (s)');
ylabel('Lateral Force (N)');
% xlim([0.5 2]);
legend('F_{yf}', 'F_{yr}', 'F_{yf,t}', 'F_{yr,t}');
title('Front-Rear Lateral forces - Transient');
grid on;

figure('Position', [100 100 700 500]);
plot(t, alphaF, LineWidth=2);
hold on;
plot(t, alphaR, LineWidth=2);
xlabel('Time (s)');
ylabel('Side-Slip Angle (deg)');
ylim([-14 2]);
legend('\alpha_f', '\alpha_r');
title('Front-Rear Side-slip angles - Transient');
grid on;

% Lateral Velocity - Yaw Rate
figure('Position', [100 100 700 500]);
plot(t, psid*180/pi, 'LineWidth', 2, Color=[0.8500, 0.3250, 0.0980]);
ylabel('Yaw Rate (deg/s)');
xlabel('Time (s)');
title('Yaw Rate - Transient');
grid on;

```

Code to evaluate lateral behaviour using vehicle model accounting for load transfer.

```
clear
close all
clc

set(0, 'DefaultAxesFontSize', 16);
set(0, 'DefaultTextFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

g = 9.81;

% Vehicle parameters
loadDist = 0.46;
load('Kya_Fz_fit_LT.mat');
l = 2.78; % wheelbase (m)
a = loadDist*l; % CG - front axle (m)
b = (1-loadDist)*l; % CG - rear axle (m)
m = 1739; % vehicle mass (kg)
Vx = 30; % vehicle speed (m/s)
I = 3508; % yaw inertia (kgm^2)
t_f = 1.6; % front track width (m)
t_r = 1.6; % front track width (m)
hG = 0.5;

% Initial conditions
zita0 = [0, 0, 0, 0, 0];

% Load tire nonlinear Fy-alpha curve
load('..\splineNorm.mat');
spline_norm = sNorm';
alpha_spline = alphaRange';

% Simulation parameters
Tsim = 100;
% Step Steer
t = 0:0.1:Tsim;
delta = interp1([ 0, 1, 1.3, Tsim],[0, 0, -12, -12],t);
dt = 1e-4;
dt_save = 1e-2;

R = 1./(delta(12:end)*pi/180);
deltaR = 1./(R+t_f/2) * 180/pi;
deltaL = 1./(R-t_f/2) * 180/pi;

% Simulation
file = 'Lateral_LoadTransfer_Nonlinear_sim.slx';
% open(file);
sim(file);

figure;
plot(Fz_fit, KyaF_scaled);
hold on;
plot(Fz_fit, KyaR_scaled);

%% Plots
% Lateral Velocity - Yaw Rate
figure('Position', [100 100 700 500]);
```

```

plot(t, psid, 'LineWidth', 2);
ylabel('Yaw Rate (rad/s)');
xlabel('Time (s)');
grid on;

% Side-slip angles
figure('Position', [100 100 700 500]);
plot(t, alphaFR, LineWidth=2);
hold on;
plot(t, alphaFL, LineWidth=2);
hold on;
plot(t, alphaRR, LineWidth=2);
hold on;
plot(t, alphaRL, LineWidth=2);
xlabel('Time (s)');
ylabel('Side-Slip Angle (deg)');
legend('\alpha_{FR}', '\alpha_{FL}', '\alpha_{RR}', '\alpha_{RL}');
grid on;

% Lateral forces
figure('Position', [100 100 700 500]);
plot(t, FyFR, LineWidth=2);
hold on;
plot(t, FyFL, LineWidth=2);
hold on;
plot(t, FyRR, LineWidth=2);
hold on;
plot(t, FyRL, LineWidth=2);
xlabel('Time (s)');
ylabel('Lateral Force (N)');
legend('F_{y,FR}', 'F_{y,FL}', 'F_{y,RR}', 'F_{y,RL}');
grid on;
ax = gca;
ax.YAxis.Exponent = 0;

% Trajectory
figure;
plot(xG, yG, LineWidth=2);
xlabel('X_G (m)');
ylabel('Y_G (m)');
grid on;
save('Trajectory_NonLinear_LoadTransfer', 'xG', 'yG');
save('Nonlinear_LoadTransfer', 't', 'psid', 'FyFR', 'FyFL', 'FyRR', 'FyRL',
'\alphaFR', '\alphaFL', '\alphaRR', '\alphaRL');

```

Code to compare lateral behaviour of vehicle models with and without load transfer contribution.

```

Cfr = Cij.data(end,1);
Cfl = Cij.data(end,2);
Crr = Cij.data(end,3);
Crl = Cij.data(end,4);

Fzfr = Fzij.data(end,1);
Fzfl = Fzij.data(end,2);
Fzrr = Fzij.data(end,3);
Fzrl = Fzij.data(end,4);

```

```

Cf = interp1(Fz_fit, KyaF_scaled, (1-loadDist)*m*g/2, 'linear');
Cr = interp1(Fz_fit, KyaR_scaled, loadDist*m*g/2, 'linear');

figure('Position', [100 100 700 500]);
plot(Fzfr, Cfr, 'bo', color='r', MarkerSize=8, LineWidth=3);
hold on;
plot((1-loadDist)*m*g/2, Cf, 'bo', color='g', MarkerSize=8, LineWidth=3);
hold on;
plot(Fzfl, Cfl, 'bo', color='r', MarkerSize=8, LineWidth=3);
hold on;
plot(Fz_fit, KyaF_scaled, LineWidth=3, Color=[0, 0.4470, 0.7410]);
xlabel('$F_z$ (N)', Interpreter='latex');
ylabel('$K_{y \alpha}$ (N/rad)', Interpreter='latex');
ylim([0, 80000]);
title('Cornering stiffness - Front');
legend('Double-track', 'Single-track');
grid on;

figure('Position', [100 100 700 500]);
plot(Fzrr, Crr, 'bo', color='r', MarkerSize=8, LineWidth=3);
hold on;
plot(loadDist*m*g/2, Cr, 'bo', color='g', MarkerSize=8, LineWidth=3);
hold on;
plot(Fzrl, Crl, 'bo', color='r', MarkerSize=8, LineWidth=3);
hold on;
plot(Fz_fit, KyaR_scaled, LineWidth=3, Color=[0, 0.4470, 0.7410]);
xlabel('$F_z$ (N)', Interpreter='latex');
ylabel('$K_{y \alpha}$ (N/rad)', Interpreter='latex');
ylim([0, 120000]);
title('Cornering stiffness - Rear');
legend('Double-track', 'Single-track');
grid on;

```

Code to compare trajectory of vehicle models with and without load transfer contribution.

```

clear
close all
clc

nonlin_LT = load("Trajectory_NonLinear_LoadTransfer.mat");
nonlin = load('..\Lateral_Nonlinear\Trajectory_NonLinear.mat');

figure('Position', [100 100 700 500]);
plot(-nonlin.yG, nonlin.xG, LineWidth=2);
hold on;
plot(-nonlin_LT.yG, nonlin_LT.xG, LineWidth=2);
xlabel('x_G (m)');
ylabel('y_G (m)');
axis equal;
legend('Nonlinear', 'Load Transfer');
title('Trajectory - Nonlinear, Load Transfer');
grid on;

```

Code to assess vertical dynamics of the 1-DOF quarter-car model.

```

clear
close all

```

```

clc

set(0, 'DefaultAxesFontSize', 16);
set(0, 'DefaultTextFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

g = 9.81;

% Bump
data1 = readmatrix('step2', 'NumHeaderLines', 2);
t = data1(3:end,2);
y = data1(3:end,1)./1000;

% Vehicle parameters
Ms = 317.5;           % sprung mass (kg)
K = 19740;            % spring stiffness (N/m)
c = 1500;             % damping coefficient (Ns/m)
F_static = Ms*g;

% Simulation
Tsim = 4;
y_dot = 0;
for kk=1:(length(y)-1)
    y_dot = [y_dot (y(kk+1)-y(kk))/(t(kk+1)-t(kk))];
end
dt = 1e-5;
dt_save = 1e-2;
file = 'Vertical_1DOF_sim.slx';
% open(file);
sim(file);

% Plots
figure('Position', [100 100 700 500]);
plot(x.time, x.data*1000, LineWidth=2);
xlabel('Time (s)');
ylabel('Displacement (mm)');
title('Sprung Mass Displacement');
grid on;

figure('Position', [100 100 700 500]);
plot(x_ddot.time, x_ddot.data*1000, LineWidth=2);
xlabel('Time (s)');
ylabel('Acceleration (mm/s^2)');
title('Sprung Mass Acceleration');
grid on;

figure('Position', [100 100 700 500]);
plot(F_tire.time, F_tire.data, LineWidth=2);
xlabel('Time (s)');
ylabel('Force (N)');
title('Vertical Force at Contact Patch');
grid on;

acc_rms = rms(x_ddot.data);
DLC = rms(F_tire.data)/F_static;
DLF = (max(F_tire.data)-min(F_tire.data))/F_static;

```

Code to evaluate frequency response of the 1-DOF quarter-car model.

```
%% TF - step
clear
close all
clc

% Vehicle parameters
Ms = 317.5;           % sprung mass (kg)
K = 19740;            % spring stiffness (N/m)
c = 1500;             % damping coefficient (Ns/m)

omegan = sqrt(K/Ms);

s = tf('s');

zeta = 0.1:0.2:1.5;
for j = 1:length(zeta)
    c = 2*zeta(j) * Ms * omegan;
    H = tf([c K], [Ms c K]);
    H = H * s^2;
    sys = ss(H);
    figure(8);
    set(gcf, 'Position', [100 100 800 600]);
    hold on;
    opts = bodeoptions;
    opts.PhaseVisible = 'off';
    opts.MagVisible = 'on';
    h = bodeplot(sys, opts);
    grid on;
end
legendStrings = arrayfun(@(zeta) sprintf('\zeta = %.0f', zeta), zeta,
    'UniformOutput', false);
legend(legendStrings);
oldLines = findall(gca, 'Type', 'line');

c = 1500;
H = tf([c K], [Ms c K]);
H = H * s^2;
sys = ss(H);
figure(8);
set(gcf, 'Position', [100 100 800 600]);
hold on;
bode(sys);
newLines = setdiff(findall(gca, 'Type', 'line'), oldLines);
set(newLines, 'LineWidth', 2);
title('Sprung Mass Acceleration');
grid on;
```

Code to assess vertical dynamics of the 2-DOFs quarter-car model and optimise suspension parameters.

```
clear
close all
clc

set(0, 'DefaultAxesFontSize', 16);
```

```

set(0, 'DefaultFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

data = readmatrix("Damper_Mid.xlsx");
v = data(:,1)/1e3;
F = data(:,2);

figure('Position', [100 100 700 500]);
plot(v, F, LineWidth=2);
xlabel('Velocity (m/s)');
ylabel('Force (N)');
title('Nonlinear damper Force-speed curve');
grid on;

% Bump
data1 = readmatrix('step2', 'NumHeaderLines', 2);
t_input = data1(3:end,2);
y_input = data1(3:end,1)./1000;

% Vehicle parameters
Ms = 317.5;           % sprung mass (kg)
Mu = 45.4;           % unsprung mass (kg)
Ks = 22000;          % spring stiffness (N/m)
Kt = 192000;         % tire stiffness (N/m)
c = 1500;            % damping coefficient (Ns/m)

% Simulation
Tsim = 4;
t = 0:0.1:Tsim;
dt = 1e-4;
dt_save = 1e-2;
file = 'Vertical_2DOFs_sim.slx';
open(file);
sim(file);

% Plots
figure('Position', [100 100 700 500]);
plot(xu.time, xu.data, linewidth=2);
xlabel('Time (s)');
ylabel('Unsprung Mass Displacement (m)');
grid on;

figure('Position', [100 100 700 500]);
plot(xs.time, xs.data, linewidth=2);
xlabel('Time (s)');
ylabel('Sprung Mass Displacement (m)');
grid on;

figure('Position', [100 100 700 500]);
plot(xu_ddot.time, xu_ddot.data, linewidth=2);
xlabel('Time (s)');
ylabel('Unsprung Mass Acceleration (m/s^2)');
grid on;

figure('Position', [100 100 700 500]);
plot(xs_ddot.time, xs_ddot.data, linewidth=2);
xlabel('Time (s)');
ylabel('Sprung Mass Acceleration (m/s^2)');
grid on;

```



```

Frms = rms(F_tire.data);
xs_ddot_rms = rms(xs_ddot.data);

xs_ddot_rms_opt = [];
Frms_opt = [];
c_factor = 0.1:0.1:1.7;
k = 18000:500:26000;
for i = 1:length(c_factor)
    F_opt = F*c_factor(i);
    for j = 1:length(k)
        Ks = k(j);
        sim(file);
        xs_ddot_rms_opt(i,j) = rms(xs_ddot.data);
        Frms_opt(i,j) = rms(F_tire.data);
    end
    figure(10);
    hold on;
    plot(v, F_opt);

end

figure(10);
set(gcf, 'Position', [100 100 800 600]);
grid on;
xlabel('Velocity (m/s)');
xlim([-0.6 0.9]);
ylabel('Force (N)');
title('Nonlinear damper Force-speed curve');
legendStrings = arrayfun(@(c) sprintf('j = %.2f', c), c_factor,
'UniformOutput', false);
legend(legendStrings);
grid on;

figure;
set(gcf, 'Position', [100 100 800 600]);
surf(k, c_factor, Frms_opt);
xlabel('Spring Stiffness (N/m)');
ylabel('Scaling Factor (-)');
zlabel('Force (N)');
title('Vertical Force RMS Optimization Map');
grid on;

figure;
set(gcf, 'Position', [100 100 800 600]);
surf(k, c_factor, xs_ddot_rms_opt);
xlabel('Spring Stiffness (N/m)');
ylabel('Scaling Factor (-)');
zlabel('Acceleration (m/s^2)');
title('Acceleration RMS Optimization Map');
grid on;

[val_Fz, idx_Fz] = min(Frms_opt(:));
[row_Fz, col_Fz] = ind2sub(size(Frms_opt), idx_Fz);

[val_acc, idx_acc] = min(xs_ddot_rms_opt(:));
[row_acc, col_acc] = ind2sub(size(xs_ddot_rms_opt), idx_acc);

```

```

figure(13);
set(gcf, 'Position', [100 100 800 600]);
plot(v, F*c_factor(idx_Fz), LineWidth=2);
hold on;
plot(v, F*c_factor(idx_acc), LineWidth=2);
xlabel('Velocity (m/s)');
ylabel('Force (N)');
title('Optimized Damper Curves');
legend('Handling', 'Comfort');
grid on;

```

Code to evaluate frequency response of the 2-DOFs quarter-car model.

```

clear
close all
clc

%% Bode
omegan = 7.878;
zeta = 0.1:0.2:1.5;
xsddot_rms = [];
Ftire_rms = [];

for j = 1:length(zeta)
    c = 2* zeta(j) * Ms * omegan;
    A = [0 0 1 0; 0 0 0 1; -(Ks+Kt)/Mu Ks/Mu -c/Mu c/Mu; Ks/Ms -Ks/Ms c/Ms
    -c/Ms];
    B = [0 0 Kt/Mu 0]';
    C1 = [Ks/Ms -Ks/Ms c/Ms -c/Ms]; % discomfort index
    D1 = 0;
    C2 = [Kt 0 0 0]; % road holding
    D2 = -Kt;

    sys1 = ss(A,B,C1,D1);
    sys2 = ss(A,B,C2,D2);

    figure(1);
    set(gcf, 'Position', [100 100 800 600]);
    hold on;
    opts = bodeoptions;
    opts.PhaseVisible = 'off';
    opts.MagVisible = 'on';
    h = bodeplot(sys1, opts);
    title('Sprung Mass Acceleration');
    grid on;

    figure(2);
    set(gcf, 'Position', [100 100 800 600]);
    hold on;
    opts = bodeoptions;
    opts.PhaseVisible = 'off';
    opts.MagVisible = 'on';
    h = bodeplot(sys2, opts);
    title('Vertical Force at Contact Patch');
    grid on;
end
legendStrings = arrayfun(@(zeta) sprintf('\zeta = %.1f', zeta), zeta,
'UniformOutput', false);

```

```

legend(legendStrings);

oldLines = findall(gca,'Type','line');

c = 1500;
A = [0 0 1 0; 0 0 0 1; -(Ks+Kt)/Mu Ks/Mu -c/Mu c/Mu; Ks/Ms -Ks/Ms c/Ms -
c/Ms];
B = [0 0 Kt/Mu 0]';
C1 = [Ks/Ms -Ks/Ms c/Ms -c/Ms]; % discomfort index
D1 = 0;
C2 = [Kt 0 0 0]; % road holding
D2 = -Kt;
sys = ss(A,B,C1,D1);
% sys = ss(A,B,C2,D2);
figure(1);
set(gcf, 'Position', [100 100 800 600]);
hold on;
bode(sys);
newLines = setdiff(findall(gca,'Type','line'), oldLines);
set(newLines,'LineWidth',2);
grid on;

```

Code to evaluate spring stiffness range during optimisation according to natural frequency.

```

clear
close all
clc

% Vehicle parameters
Ms = 317.5; % sprung mass (kg)
Mu = 45.4; % unsprung mass (kg)
Ks = 18000:1000:26000; % spring stiffness (N/m)
Kt = 192000; % tire stiffness (N/m)
c = 1500; % damping coefficient (Ns/m)

a = Ms * Mu;
b = -(Mu*Ks+Ms*Ks+Ms*Kt);
c = Ks*Kt;

omega1 = sqrt((-b+sqrt(b.^2-4*a.*c))/(2*a)) /2/pi;
omega2 = sqrt((-b-sqrt(b.^2-4*a.*c))/(2*a)) /2/pi;

```

Code to compare original and optimised response of the 2-DOFs quarter-car model.

```

clear
close all
clc

set(0, 'DefaultAxesFontSize', 16);
set(0, 'DefaultTextFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

data = readmatrix("Damper_Mid.xlsx");
v = data(:,1)/1e3;
F = data(:,2);

```

```

data1 = readmatrix('step2', 'NumHeaderLines', 2);
t_input = data1(3:end,2);
y_input = data1(3:end,1)./1000;

g = 9.81;
% Vehicle parameters
Ms = 317.5;           % sprung mass (kg)
Mu = 45.4;           % unsprung mass (kg)
Ks = 18000;          % spring stiffness (N/m)
Kt = 192000;         % tire stiffness (N/m)
c = 1500;            % damping coefficient (Ns/m)
F_static = Ms*g;

% Simulation
Tsim = 4;
t = 0:0.1:Tsim;
dt = 1e-4;
dt_save = 1e-2;
file = 'Vertical_2DOFs_sim.slx';
% open(file);

c_factor = [1 0.45];
for i = 1:length(c_factor)
    F_opt = F*c_factor(i);
    sim(file);

    figure(1);
    set(gcf, 'Position', [100 100 800 600]);
    hold on;
    plot(xs_ddot.time, xs_ddot.data, linewidth=2);
    xlabel('Time (s)');
    ylabel('Acceleration (m/s^2)');
    title('Sprung Mass Acceleration');
    grid on;
    acc_rms(i) = rms(xs_ddot.data);

    figure(2);
    set(gcf, 'Position', [100 100 800 600]);
    hold on;
    plot(F_tire.time, F_tire.data, Linewidth=2);
    xlabel('Time (s)');
    ylabel('Force (N)');
    title('Vertical Force at Contact Patch');
    grid on;
    DLC(i) = rms(F_tire.data)/F_static;
    DLF(i) = (max(F_tire.data)-min(F_tire.data))/F_static;
end
legend('Original', 'Optimized');

```

Code to assess vertical dynamics of the half-car model and optimise suspension parameters.

```

clear
close all
clc

set(0, 'DefaultAxesFontSize', 16);

```

```

set(0, 'DefaultFontSize', 16);
set(0, 'DefaultLegendFontSize', 16);

% Nonlinear damper
data = readmatrix("../Vertical_2DOFs/Damper_Mid.xlsx");
v = data(:,1)/1e3;
F = data(:,2);

% Bump
data1 = readmatrix('../Vertical_1DOF/step2', 'NumHeaderLines', 2);
t_input = data1(3:end,2);
y_input = data1(3:end,1)./1000;

g = 9.81;
% Vehicle parameters
a = 1.25;          % CG - front axle distance (m)
b = 1.51;          % CG - rear axle distance (m)
l = 2.76;          % wheelbase (m)
I = 1222;          % pitch inertia (kgm^2)
ms = 690;          % half vehicle body mass (kg)
muf = 40.5;        % front wheel mass (kg)
mur = 45.4;        % rear wheel mass (kg)
ksf = 18000;        % front suspension stiffness (N/m)
ksr = 18000;        % rear suspension stiffness (N/m)
csf = 1500;         % front damping coefficient (Ns/m)
csr = 1500;         % rear damping coefficient (Ns/m)
ktf = 192000;       % front tyre stiffness (N/m)
ktr = 192000;       % rear tyre stiffness (N/m)
Vx = 30;           % vehicle longitudinal speed (m/s)

F_static_f = (ms*b/l + muf)*g;
F_static_r = (ms*a/l + mur)*g;

delay = 1/Vx;

Gr = 6.4*10^-7;    % Class B road

% Simulation
Tsim = 10;
t = 0:0.1:Tsim;
dt = 1e-4;
dt_save = 1e-2;

c_factor = [1 0.55];
for i = 1:length(c_factor)
    F_opt = F*c_factor(i);
    file = 'Vertical_HalfCar_sim.slx';
    % open(file);
    sim(file);

    % Plots
    figure(1);
    set(gcf, 'Position', [100 100 800 600]);
    hold on;
    plot(xuf.time, xur.data, linewidth=2);
    xlabel('Time (s)');
    ylabel('Displacement (m)');
    title('Front Unsprung Mass Displacement');
    legend('Original', 'Optimized');

```

```

grid on;

figure(2);
set(gcf, 'Position', [100 100 800 600]);
hold on;
plot(xur.time, xur.data, linewidth=2);
xlabel('Time (s)');
ylabel('Displacement (m)');
title('Rear Unsprung Mass Displacement');
legend('Original', 'Optimized');
grid on;

figure(3);
set(gcf, 'Position', [100 100 800 600]);
hold on;
plot(xs.time, xs.data, linewidth=2);
xlabel('Time (s)');
ylabel('Displacement (m)');
title('Sprung Mass Displacement');
legend('Original', 'Optimized');
grid on;

figure(4);
set(gcf, 'Position', [100 100 800 600]);
hold on;
plot(theta.time, theta.data, linewidth=2);
xlabel('Time (s)');
ylabel('Angle (deg)');
title('Sprung Mass Pitch Angle');
legend('Original', 'Optimized');
grid on;

figure(5);
set(gcf, 'Position', [100 100 800 600]);
hold on;
plot(theta_ddot.time, theta_ddot.data, linewidth=2);
xlabel('Time (s)');
ylabel('Angular Acceleration (deg/s^2)');
title('Sprung Mass Pitch Acceleration');
legend('Original', 'Optimized');
grid on;

figure(8);
set(gcf, 'Position', [100 100 800 600]);
hold on;
plot(xs_ddot.time, xs_ddot.data, linewidth=2);
xlabel('Time (s)');
ylabel('Acceleration (m/s^2)');
title('Sprung Mass Vertical Acceleration');
legend('Original', 'Optimized');
grid on;

figure(9);
set(gcf, 'Position', [100 100 800 600]);
hold on;
plot(F_tire_f.time, F_tire_f.data, linewidth=2);
xlabel('Time (s)');
ylabel('Force (N)');
title('Front Tyre Force at Contact Patch');

```

```

legend('Original', 'Optimized');
grid on;

figure(10);
set(gcf, 'Position', [100 100 800 600]);
hold on;
plot(F_tire_r.time, F_tire_r.data, linewidth=2);
xlabel('Time (s)');
ylabel('Force (N)');
title('Rear Tyre Force at Contact Patch');
legend('Original', 'Optimized');
grid on;

acc_rms(i) = rms(xs_ddot.data);
pitch_rms(i) = rms(theta_ddot.data);
DLCf(i) = rms(F_tire_f.data)/F_static_f;
DLCr(i) = rms(F_tire_r.data)/F_static_r;
DLFf(i) = (max(F_tire_f.data)-min(F_tire_f.data))/F_static_f;
DLFr(i) = (max(F_tire_r.data)-min(F_tire_r.data))/F_static_r;
end
legend('Original', 'Optimized');

```

Code to evaluate frequency response of the half-car model.

```

clear
close all
clc

%% Bode
c = 5:500:3505;

for i=1:length(c)
    A = [0 0 0 0 1 0 0 0;
          0 0 0 0 0 1 0 0;
          0 0 0 0 0 0 1 0;
          0 0 0 0 0 0 0 1;
          -ksf/ms-ksr/ms ksf*a/ms-ksr*b/ms ksf/ms ksr/ms -c(i)/ms-c(i)/ms
          c(i)*a/ms-c(i)*b/ms c(i)/ms c(i)/ms;
          ksf*a/I-ksr*b/I -ksf*a^2/I-ksr*b^2/I -ksf*a/I ksr*b/I c(i)*a/I-
          c(i)*b/I -c(i)*a^2/I-c(i)*b^2/I -c(i)*a/I c(i)*b/I;
          ksf/muf -ksf*a/muf -ksf/muf-ktf/muf 0 c(i)/muf -c(i)*a/muf -
          c(i)/muf 0;
          ksr/mur ksr*b/mur 0 -ksr/mur-ktr/mur c(i)/mur c(i)*b/mur 0 -
          c(i)/mur];
    B1 = [0 0;
           0 0;
           0 0;
           0 0;

           0;
           0;
           0;
           0;
           0;
           ktf/muf;
           0];

```

```

C1 = [-ksf/ms-ksr/ms ksf*a/ms-ksr*b/ms ksf/ms ksr/ms -c(i)/ms-c(i)/ms
c(i)*a/ms-c(i)*b/ms c(i)/ms c(i)/ms]; % vertical discomfort index
D1 = [0];
sys = ss(A,B1,C1,D1);
figure(1);
set(gcf, 'Position', [100 100 800 600]);
hold on;
opts = bodeoptions;
opts.PhaseVisible = 'off';
opts.MagVisible = 'on';
h = bodeplot(sys, opts);
title('Sprung Mass Vertical Acceleration', 'FontSize', 16);
grid on;

C2 = [ksf*a/I-ksr*b/I -ksf*a^2/I-ksr*b^2/I -ksf*a/I ksr*b/I c(i)*a/I-
c(i)*b/I -c(i)*a^2/I-c(i)*b^2/I -c(i)*a/I c(i)*b/I]; % pitch
discomfort index
D2 = [0];
sys = ss(A,B2,C2,D2);
figure(2);
set(gcf, 'Position', [100 100 800 600]);
hold on;
opts = bodeoptions;
opts.PhaseVisible = 'off';
opts.MagVisible = 'on';
h = bodeplot(sys, opts);
title('Sprung Mass Pitching Acceleration', 'FontSize', 16);
grid on;

end
oldLines = findall(gca, 'Type', 'line');
c = 0:500:3500;
legendStrings = arrayfun(@(c) sprintf('c = %.0f', c), c, 'UniformOutput',
false);
legend(legendStrings);

csf = 1500;
csr = 1500;
A = [0 0 0 0 1 0 0 0;
0 0 0 0 0 1 0 0;
0 0 0 0 0 0 1 0;
0 0 0 0 0 0 0 1;
-ksf/ms-ksr/ms ksf*a/ms-ksr*b/ms ksf/ms ksr/ms -c(i)/ms-c(i)/ms
c(i)*a/ms-c(i)*b/ms c(i)/ms c(i)/ms;
ksf*a/I-ksr*b/I -ksf*a^2/I-ksr*b^2/I -ksf*a/I ksr*b/I c(i)*a/I-
c(i)*b/I -c(i)*a^2/I-c(i)*b^2/I -c(i)*a/I c(i)*b/I;
ksf/muf -ksf*a/muf -ksf/muf-ktf/muf 0 c(i)/muf -c(i)*a/muf -c(i)/muf
0;
ksr/mur ksr*b/mur 0 -ksr/mur-ktr/mur c(i)/mur c(i)*b/mur 0 -c(i)/mur];
B = [0;
0;
0;
0;
0;
0;
ktf/muf;
0];

```



```

C1 = [-ksf/ms-ksr/ms ksf*a/ms-ksr*b/ms ksf/ms ksr/ms -c(i)/ms-c(i)/ms
c(i)*a/ms-c(i)*b/ms c(i)/ms c(i)/ms]; % vertical discomfort index
D1 = [0];
sys = ss(A,B1,C1,D1);
figure(1);
set(gcf, 'Position', [100 100 800 600]);
xlim([1 1000]);
ylim([-40 80]);
hold on;
bode(sys);
title('Sprung Mass Vertical Acceleration');
grid on;
newLines = setdiff(findall(gca,'Type','line'), oldLines);
set(newLines,'LineWidth',2, Color=[0.8500, 0.3250, 0.0980]);
grid on;

C2 = [ksf*a/I-ksr*b/I -ksf*a^2/I-ksr*b^2/I -ksf*a/I ksr*b/I c(i)*a/I-
c(i)*b/I -c(i)*a^2/I-c(i)*b^2/I -c(i)*a/I c(i)*b/I]; % pitch
discomfort index
D2 = [0];
sys = ss(A,B2,C2,D2);
figure(2);
set(gcf, 'Position', [100 100 800 600]);
hold on;
bode(sys);
title('Sprung Mass Pitching Acceleration', 'FontSize', 16);
grid on;
newLines = setdiff(findall(gca,'Type','line'), oldLines);
set(newLines,'LineWidth',2, Color=[0.8500, 0.3250, 0.0980]);
grid on;

```

Code to perform derivatives analysis.

```

clear
close all
clc

% Vehicle parameters
load('Lateral_Linear\Load_distribution.mat');
load('Lateral_Linear\CornStiff_NissanGTR.mat');
Cf = Cf*180/pi; % front cornering stiffness (N/rad)
Cr = Cr*180/pi; % rear cornering stiffness (N/rad)
l = 2.78; % wheelbase (m)
a = loadDist*l; % CG - front axle (m)
b = (1-loadDist)*l; % CG - rear axle (m)
m = 1739; % vehicle mass (kg)
Vx = 10; % vehicle speed (m/s)
I = 2130; % yaw inertia (kgm^2)

% Derivatives of stability
Nd = a*Cf;
Yd = Cf;
Nr = -(1/Vx)*(a^2*Cf+b^2*Cr);
Yb = -(Cf+Cr);
Nb = -(a*Cf-b*Cr);
Yr = -(1/Vx)*(a*Cf-b*Cr);

% Step response
Tsim = 5;

```

```

t = linspace(0,Tsim,100);
delta = interp1([ 0, 1, 1.3, Tsim],[0, 0, -12*pi/180, -12*pi/180],t);

k = Nb + (Yb*Nr)/(m*Vx) - (Nb*Yr)/(m*Vx);
omegan = sqrt(k/I);
Cc = 2*I*omegan;
c = -(Nr + (I*Yb)/(m*Vx));
zeta = c/Cc;
omegad = sqrt(1-zeta^2)*omegan;
C2 = (Yd*Nb)/(m*Vx) - (Yb*Nd)/(m*Vx);
phi = atan(omegad/(zeta*omegan));
X = (-C2*delta(end))/(k*sin(phi));

r = X.*exp(-t.*zeta.*omegan) .* sin(t.*omegad+phi) + C2./k.*delta(end);
rinf = C2*delta(end)/k;

figure('Position', [100 100 700 500]);
plot(t, r*180/pi, LineWidth=2);
hold on;
yline(rinf * 180/pi, '--', LineWidth=2, Color=[0.8500, 0.3250, 0.0980]);
xlabel('Time (s)');
ylabel('Yaw Rate (deg/s)');
title('Yaw Rate - Derivatives of Stability');
legend('Derivative analysis', 'Steady-state analytical value');
grid on;

```

Appendix I

As also shown in Appendix H, the frequency response analysis of the vertical dynamics models was carried out in MATLAB, as this software provides a more straightforward and accurate evaluation of frequency response compared to ADAMS through the implementation of a swept-sine excitation. MATLAB computes the frequency response using analytical, equation-based methods, whereas ADAMS relies on numerical simulation techniques. To evaluate the frequency response, the bode command was used, which automatically computes the magnitude and phase of the system output. This command requires the system to be expressed through a transfer function or in state-space form, i.e.:

$$\begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases} \quad (24)$$

This appendix presents the state-space representations or transfer functions of the 1-DOF model, the 2-DOF quarter-car model and the 4-DOF half-car model used in this work and describes how they were derived starting from the equations of motion.

1-DOF Quarter-Car Model

$$m\ddot{x} + c(\dot{x} - \dot{y}) + k(x - y) = 0 \rightarrow m\ddot{x} + c\dot{x} - c\dot{y} + kx - ky = 0 \rightarrow \text{Laplace (IC=0)}: ms^2X + c sX - c sY + kX - kY = 0$$

$$\rightarrow (ms^2 + cs + k)X = (cs + k)Y \rightarrow \frac{X}{Y} = \frac{cs + k}{ms^2 + cs + k} = H \rightarrow \text{acceleration} = H \cdot s^2$$

2-DOFs Quarter-Car Model

$$\begin{cases} m_{uu}\ddot{x}_u + c(\dot{x}_u - \dot{x}_s) + k_s(x_u - x_s) + k_t(x_u - y) = 0 \\ m_s\ddot{x}_s + c(\dot{x}_s - \dot{x}_u) + k_s(x_s - x_u) = 0 \end{cases} \rightarrow \begin{cases} \ddot{x}_u = -\frac{c}{m_{uu}}(\dot{x}_u - \dot{x}_s) - \frac{k_s}{m_{uu}}(x_u - x_s) - \frac{k_t}{m_{uu}}(x_u - y) \\ \ddot{x}_s = -\frac{c}{m_s}(\dot{x}_s - \dot{x}_u) - \frac{k_s}{m_s}(x_s - x_u) \end{cases}$$

$$\rightarrow \begin{cases} \ddot{x}_u = -\frac{c}{m_{uu}}\dot{x}_u + \frac{c}{m_{uu}}\dot{x}_s - \frac{k_s}{m_{uu}}x_u + \frac{k_s}{m_{uu}}x_s - \frac{k_t}{m_{uu}}x_u + \frac{k_t}{m_{uu}}y \\ \ddot{x}_s = -\frac{c}{m_s}\dot{x}_s + \frac{c}{m_s}\dot{x}_u - \frac{k_s}{m_s}x_s + \frac{k_s}{m_s}x_u \end{cases}$$

$$\begin{pmatrix} \dot{x}_u \\ \dot{x}_s \\ \ddot{x}_u \\ \ddot{x}_s \end{pmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{k_s + k_t}{m_{uu}} & \frac{k_s}{m_{uu}} & -\frac{c}{m_{uu}} & \frac{c}{m_{uu}} \\ \frac{k_s}{m_s} & -\frac{k_s}{m_s} & \frac{c}{m_s} & -\frac{c}{m_s} \end{bmatrix} \begin{pmatrix} x_u \\ x_s \\ \dot{x}_u \\ \dot{x}_s \end{pmatrix} + \begin{bmatrix} 0 \\ 0 \\ \frac{k_t}{m_{uu}} \\ 0 \end{bmatrix} \{y\}$$

Half-Car Model

$$\begin{cases}
 m_s \ddot{x}_s + c_{sf} \dot{x}_s - c_{sf} \dot{x}_{uf} - c_{sf} a \ddot{\theta} + c_{sr} \dot{x}_s - c_{sr} \dot{x}_{un} + c_{sr} b \ddot{\theta} + k_{sf} x_s - k_{sf} x_{uf} - k_{sf} a \theta + k_{sr} x_s - k_{sr} x_{un} + k_{sr} b \theta = 0 \\
 I_y \ddot{\theta} - a c_{sf} \dot{x}_s + a c_{sf} \dot{x}_{uf} + a^2 c_{sf} \ddot{\theta} + b c_{sr} \dot{x}_s - b c_{sr} \dot{x}_{un} + b^2 c_{sr} \ddot{\theta} - a k_{sf} x_s + a k_{sf} x_{uf} + a^2 k_{sf} \theta + b k_{sr} x_s - b k_{sr} x_{un} + b^2 k_{sr} \theta = 0 \\
 m_{uf} \ddot{x}_{uf} - c_{sf} \dot{x}_s + c_{sf} \dot{x}_{uf} + c_{sf} a \ddot{\theta} - k_{sf} x_s + k_{sf} x_{uf} + k_{sf} a \theta + k_e x_{uf} - k_e y_f = 0 \\
 m_{un} \ddot{x}_{un} - c_{sr} \dot{x}_s + c_{sr} \dot{x}_{un} - c_{sr} b \ddot{\theta} - k_{sr} x_s + k_{sr} x_{un} - k_{sr} b \theta + k_{en} x_{un} - k_{en} y_e = 0
 \end{cases}$$

$$\begin{cases}
 \ddot{x}_s = -\frac{c_{sf}}{m_s} \dot{x}_s + \frac{c_{sf}}{m_s} \dot{x}_{uf} + \frac{c_{sf} a}{m_s} \ddot{\theta} - \frac{c_{sr}}{m_s} \dot{x}_s + \frac{c_{sr}}{m_s} \dot{x}_{un} - \frac{c_{sr} b}{m_s} \ddot{\theta} - \frac{k_{sf}}{m_s} x_s + \frac{k_{sf}}{m_s} x_{uf} + \frac{k_{sf} a}{m_s} \theta - \frac{k_{sr}}{m_s} x_s + \frac{k_{sr}}{m_s} x_{un} - \frac{k_{sr} b}{m_s} \theta \\
 \ddot{\theta} = \frac{a c_{sf}}{I} \dot{x}_s - \frac{a c_{sf}}{I} \dot{x}_{uf} - \frac{a^2 c_{sf}}{I} \ddot{\theta} - \frac{b c_{sr}}{I} \dot{x}_s + \frac{b c_{sr}}{I} \dot{x}_{un} - \frac{b^2 c_{sr}}{I} \ddot{\theta} + \frac{a k_{sf}}{I} x_s - \frac{a k_{sf}}{I} x_{uf} - \frac{a^2 k_{sf}}{I} \theta - \frac{b k_{sr}}{I} x_s + \frac{b k_{sr}}{I} x_{un} - \frac{b^2 k_{sr}}{I} \theta = 0 \\
 \ddot{x}_{uf} = \frac{c_{sf}}{m_{uf}} \dot{x}_s - \frac{c_{sf}}{m_{uf}} \dot{x}_{uf} - \frac{c_{sf} a}{m_{uf}} \ddot{\theta} + \frac{k_{sf}}{m_{uf}} x_s - \frac{k_{sf}}{m_{uf}} x_{uf} - \frac{k_{sf} a}{m_{uf}} \theta - \frac{k_e}{m_{uf}} x_{uf} + \frac{k_e}{m_{uf}} y_f \\
 \ddot{x}_{un} = \frac{c_{sr}}{m_{un}} \dot{x}_s - \frac{c_{sr}}{m_{un}} \dot{x}_{un} + \frac{c_{sr} b}{m_{un}} \ddot{\theta} + \frac{k_{sr}}{m_{un}} x_s - \frac{k_{sr}}{m_{un}} x_{un} + \frac{k_{sr} b}{m_{un}} \theta - \frac{k_{en}}{m_{un}} x_{un} + \frac{k_{en}}{m_{un}} y_e
 \end{cases}$$

$$\begin{pmatrix} \ddot{x}_s \\ \ddot{\theta} \\ \ddot{x}_{uf} \\ \ddot{x}_{un} \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} + \begin{pmatrix} x_5 \\ x_6 \\ x_7 \\ x_8 \end{pmatrix} + \begin{pmatrix} y_f \\ y_e \end{pmatrix}$$

$$\begin{pmatrix} \ddot{x}_s \\ \ddot{\theta} \\ \ddot{x}_{uf} \\ \ddot{x}_{un} \end{pmatrix} = \begin{pmatrix} -\frac{k_{sf}}{m_s} - \frac{k_{sr}}{m_s} & \frac{k_{sf} a}{m_s} - \frac{k_{sr} b}{m_s} & \frac{k_{sf}}{m_s} & \frac{k_{sr}}{m_s} & -\frac{c_{sf}}{m_s} - \frac{c_{sr}}{m_s} & \frac{c_{sf} a}{m_s} - \frac{c_{sr} b}{m_s} & \frac{c_{sf}}{m_s} & \frac{c_{sr}}{m_s} \\ \frac{a k_{sf}}{I} - \frac{b k_{sr}}{I} & -\frac{a^2 k_{sf}}{I} - \frac{b^2 k_{sr}}{I} & -\frac{a k_{sf}}{I} & \frac{b k_{sr}}{I} & \frac{a c_{sf}}{I} - \frac{b c_{sr}}{I} & -\frac{a^2 c_{sf}}{I} - \frac{b^2 c_{sr}}{I} & -\frac{a c_{sf}}{I} & \frac{b c_{sr}}{I} \\ \frac{k_{sf}}{m_{uf}} & -\frac{k_{sf} a}{m_{uf}} & -\frac{k_{sf}}{m_{uf}} - \frac{k_{sf}}{m_{uf}} & 0 & \frac{c_{sf}}{m_{uf}} & -\frac{c_{sf} a}{m_{uf}} & -\frac{c_{sf}}{m_{uf}} & 0 \\ \frac{k_{sr}}{m_{un}} & \frac{k_{sr} b}{m_{un}} & 0 & -\frac{k_{sr}}{m_{un}} - \frac{k_{en}}{m_{un}} & \frac{c_{sr}}{m_{un}} & \frac{c_{sr} b}{m_{un}} & 0 & -\frac{c_{en}}{m_{un}} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \frac{k_e}{m_{uf}} & 0 \\ 0 & \frac{k_{en}}{m_{un}} \end{pmatrix}$$